

Министерство образования Калининградской области
государственное автономное учреждение
Калининградской области
профессиональная образовательная организация
«Колледж предпринимательства»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ (ДИПЛОМНАЯ) РАБОТА

Тема: Разработка интернет-магазина товаров для дома

Выпускная квалификационная
(дипломная) работа
допущена к защите
Заместитель директора по УМР

_____ Бурыкина Ю.И.
«__» _____ 2026 г.

Выполнил:
обучающийся группы ИСП 22-22
специальность 09.02.07
Информационные системы и
программирование

_____ Хурсин М.А.

Руководитель:

_____ Зверев М.В.

Калининград
2026

СОДЕРЖАНИЕ

СПИСОК ТЕРМИНОВ И СОКРАЩЕНИЙ	3
ВВЕДЕНИЕ.....	4
1. ТЕХНОЛОГИИ СОЗДАНИЯ ИНТЕРНЕТ-МАГАЗИНА	6
1.1 Клиентская часть.....	7
1.2 Серверная часть.....	11
1.3 База данных	15
1.4 Программные средства для разработки.....	19
2. РАЗРАБОТКА ИНТЕРНЕТ-МАГАЗИНА	24
2.1 Централизованные системные модули	24
2.2 Процесс регистрации и авторизации пользователей.....	30
2.3 Механизм поиска и визуализации каталога товаров	41
2.4 Обзор и управление wybranными товарами и заказами	49
2.5 Панель управления заказами для курьеров	55
2.6 Административная панель для управления.....	60
2.7 Многофункциональная платформа поддержки.....	69
ЗАКЛЮЧЕНИЕ	73
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....	74

СПИСОК ТЕРМИНОВ И СОКРАЩЕНИЙ

JavaScript — язык программирования для создания интерактивных элементов на веб-страницах.

AJAX — технология для обмена данными с сервером без перезагрузки страницы.

PHP — серверный язык программирования для разработки динамических веб-приложений.

SQL — язык запросов для управления базами данных.

MySQL — популярная система управления базами данных на базе SQL.

PDO — интерфейс PHP для безопасного взаимодействия с базами данных, предотвращающий SQL-инъекции.

SQL-инъекции — атаки, при которых злоумышленник внедряет вредоносный SQL-код в запросы.

XSS-атаки — атаки, при которых злоумышленники вставляют вредоносный скрипт в веб-страницы.

CSRF-атаки — атаки, когда злоумышленник заставляет пользователя выполнить нежелательные действия на сайте.

phpMyAdmin — веб-инструмент для управления базами данных MySQL через браузер.

Open Server Panel — локальный сервер для разработки веб-приложений.

Git — система контроля версий для отслеживания изменений в коде.

MIME — интернет-стандарт, который описывает формат передачи файлов, изображений, аудио и видео через электронную почту и интернет.

ВВЕДЕНИЕ

В последние десятилетия электронная коммерция стала одной из наиболее динамично развивающихся отраслей экономики. Развитие технологий, широкое распространение интернета и мобильных устройств открыло новые возможности для бизнеса — создание и развитие интернет-магазинов. В условиях современного рынка наличие собственного онлайн-приложения продаж стало не просто конкурентным преимуществом, а необходимостью для успешного ведения бизнеса, особенно в условиях высокой конкуренции и потребности клиентов в быстром и удобном обслуживании.

Создание интернет-магазина — это сложный и многогранный процесс, включающий, как техническую, так и организационную подготовку. На начальных этапах важно определить концепцию и целевую аудиторию, выбрать подходящую платформу, а также организовать надежную инфраструктуру для безопасных операций и взаимодействия с клиентами. Разработка интернет-магазина предполагает интеграцию различных компонентов: системы для обработки заказов, платежных систем, системы учета, а также дизайн, который обеспечивает удобство навигации и приятное пользовательское взаимодействие.

Актуальность разработки интернет-магазина существенно возросла, ведь в условиях современного рынка наличие собственного онлайн-приложения для продаж становится не только конкурентным преимуществом, но и критически важным элементом выживания и успешного развития бизнеса. В эпоху высокой конкуренции и глобализации, быстрое развитие технологий и активное внедрение цифровых решений создали новую реальность — большинство компаний стремится инвестировать в создание и совершенствование своих интернет-магазинов, чтобы обслуживать клиентов в онлайн-пространстве.

В современную эпоху онлайн-торговля демонстрирует впечатляющие показатели роста, всё больше людей предпочитают приобретать товары и услуги через интернет, что обусловлено возможностью сравнить цены, выбрать наиболее подходящий товар и оформить заказ в удобное время. Это создает огромные возможности для предпринимателей, но в то же время требует особых знаний и навыков для эффективной реализации проекта — начиная с анализа рынка, заканчивая маркетинговыми стратегиями и технической реализацией.

Цель данной работы — раскрыть основные этапы и особенности процесса создания интернет-магазина, рассмотреть его структуру, выбрать оптимальные инструменты и технологии. В рамках исследования особое внимание уделяется техническим аспектам — выбору платформ, оптимизации сайта, обеспечению безопасности и удобству пользователей, а также организационным моментам, связанным с управлением и продвижением онлайн-магазина.

В условиях острой конкуренции рынку нужны инновационные решения, индивидуальный подход к клиентам и способность быстро адаптироваться к изменениям. В результате, создание и развитие интернет-магазина становится важным и актуальным направлением в сфере бизнеса, требующим системного подхода, творческого мышления и современных технологий.

1. ТЕХНОЛОГИИ СОЗДАНИЯ ИНТЕРНЕТ-МАГАЗИНА

Создание полноценного, безопасного и удобного интернет-магазина требует применения разнообразных технологий, языков программирования и специализированных инструментов, обеспечивающих надежность и функциональность платформы. В современном подходе важно также использовать адаптивный дизайн, чтобы сайт был удобен для пользователей на различных устройствах, и внедрять аналитические системы для отслеживания поведения клиентов, что помогает оптимизировать работу и повышать конверсию.

В основе такой системы лежит надежная интеграция двух основных компонентов. Первый — клиентская часть, которая обеспечивает интерактивное взаимодействие с пользователями, позволяя им легко и удобно вводить данные, получать ответы и управлять процессами. Второй — серверная часть, которая занимается обработкой всей поступающей информации, выполнением бизнес-логики, хранением данных и управлением системой в целом. Совместная работа этих компонентов обеспечивает эффективное и стабильное функционирование системы.

Для реализации этих элементов используются современные веб-технологии для формирования интерфейса, а также языки программирования на серверной стороне. Не менее важную роль играет база данных, которая хранит сведения о товарах, заказах, пользователях и прочих элементах интернет-магазина, позволяя быстро получать и обновлять информацию в реальном времени.

Также в такой системе особое значение уделяется вопросам безопасности. В связи с обработкой персональных данных, платёжной информации и конфиденциальных данных клиентов, разработка интернет-магазина требует внедрения надежных мер защиты. Не менее важно защищать систему от разнообразных атак, таких как SQL-инъекции и XSS-атаки, которые могут привести к потере данных пользователей или к утрате

конфиденциальности данных, связанных с информационными системами. Кроме того, внедрение методов шифрования и строгие процедуры аутентификации помогают обеспечить дополнительную защиту и доверие клиентов. Кроме технических аспектов, крайне важны пользовательский опыт и дизайн сайта:

- ✓ удобная навигация;
- ✓ привлекательный и интуитивно понятный интерфейс;
- ✓ быстрая загрузка страниц;
- ✓ множество вариантов оплаты и доставки.

Объединение всех этих аспектов создает положительный пользовательский опыт, что не только способствует их рекомендацию магазина другим людям, но и укрепление репутации бизнеса в целом, ведь все эти элементы существенно влияют на удовлетворенность посетителей.

1.1 Клиентская часть

Для клиентской части отлично подходит JavaScript. Это универсальный язык программирования, который выполняется прямо в браузере на стороне клиента. Он превращает статический сайт в интерактивную платформу, которая подстраивается под действия пользователя. Благодаря его возможностям происходит ускорение взаимодействия, повышение удобства и создание привлекательного визуального оформления, что делает пользователей более довольными и помогает бизнесу достигать своих целей. Он обеспечивает мгновенную реакцию сайта на действия пользователя, такие как:

- ✓ заполнение и отправка форм;
- ✓ выбор фильтров и сортировка данных;
- ✓ обеспечение плавных анимаций и эффектов;
- ✓ динамическое обновление контента без перезагрузки.

Это создает эффект отзывчивого сайта, где реакции происходят без долгой перезагрузки страницы, что значительно ускоряет взаимодействие и

повышает удобство использования. Также благодаря ему можно реализовать разнообразные элементы интерфейса, делая работу с сайтом проще и приятнее. В результате пользователь получает более плавное и интуитивное взаимодействие, а у разработчика появляется возможность создавать более сложный и привлекательный интерфейс.

Также, используя JavaScript, можно реализовать обмен данными с сервером в фоновом режиме, без необходимости перезагружать всю страницу целиком. Этот метод взаимодействия называется AJAX, и он позволяет браузеру отправлять запросы к серверу и получать ответы асинхронно, то есть независимо от работы основного интерфейса страницы. Благодаря этому подходу можно создавать более динамичные и отзывчивые страницы, поскольку содержимое сайта обновляется автоматически, без лишних задержек и перерывов. В результате, пользователи могут получать новые данные, отправлять информацию или взаимодействовать с сайтом более комфортно, ощущая плавность и непрерывность работы интерфейса, что значительно улучшает общее впечатление и удобство использования. Яркими примерами использования AJAX являются:

- ✓ обновление только определённой части страницы без необходимости перезагружать всю страницу;
- ✓ автоматическая подгрузка новой информации через определённые промежутки времени;
- ✓ отправка данных через формы и обработка ответов от сервера без обновления страницы.

Эта технология значительно повышает удобство и скорость работы сайта, делая его более современным и профессиональным. Благодаря AJAX пользователь получает более плавное взаимодействие, уменьшается время ожидания, а разработчики получают возможность создавать богатые и интерактивные пользовательские интерфейсы, не вызывая чрезмерную нагрузку на сервер. Это способствует лучшему восприятию сайта и увеличению его эффективности в выполнении задач пользователя.

А также JavaScript широко используется для реализации различных визуальных эффектов, которые делают сайт более привлекательным и современным. Благодаря этому язык получается создать динамичный и отзывчивый интерфейс, что заметно улучшает пользовательский опыт. Визуальные эффекты помогают сделать взаимодействие с сайтом более интуитивным. Также он обеспечивает удобство навигации и повышение восприятия сайта через такие возможности, как:

- ✓ интерактивные элементы, например, динамическое раскрытие меню, модальные окна, слайдеры, галереи изображений;
- ✓ формы с валидацией и автозаполнением в реальном времени и подсказки при заполнении;
- ✓ анимации и эффекты при прокрутке, которые делают просмотр страниц более захватывающим.

Это помогает создать уникальный и запоминающийся дизайн, который не только выделяет сайт среди конкурентов, но и вызывает у пользователей желание возвращаться снова и снова благодаря своему стилю, удобству и проявляемой заботе о комфорте и впечатлении пользователей. А также такой дизайн укрепляет доверие, делает взаимодействие приятным и стимулирует долгосрочную лояльность пользователей.

Также неотъемлемым элементом клиентской части является адаптивный дизайн, который играет ключевую роль в обеспечении лучшего пользовательского опыта. Он позволяет отображать содержимое страницы правильно и удобно для восприятия на различных устройствах — от небольших мобильных телефонов до крупных настольных мониторов. Благодаря этому пользователи получают одинаково качественный и удобный доступ к информации вне зависимости от того, какое устройство они используют.

Основной принцип при разработке адаптивного интерфейса является то, что сначала создается дизайн для смартфонов и мобильных устройств, а затем его расширяют и дорабатывают для более больших экранов. Такой подход

обеспечивает более эффективное использование ресурсов, проще управляется и позволяет лучше учитывать особенности мобильных устройств, такие как ограниченный экран и меньшие размеры компонентов.

Для достижения универсальности и высокой совместимости, обеспечивающие равный уровень удобства на всех устройствах, чтобы обеспечить максимально плавное и эффективное отображение контента на разнообразных платформах, применяются следующие техники:

- ✓ использование современных гибких сеток и пропорциональных единиц измерения;
- ✓ оптимизация изображений для меньших размеров и высокой скорости загрузки;
- ✓ создание увеличенных зон взаимодействия для мобильных и планшетных устройств;
- ✓ тестирование интерфейса на различных разрешениях и устройствах для выявления потенциальных проблем;
- ✓ применение семантической верстки и доступных технологий для повышения удобства навигации;
- ✓ минимизация количества контента, чтобы избежать перегрузки интерфейса на небольших экранах.

Кроме того, важной частью является тестирование интерфейса на различных устройствах и браузерах, что позволяет выявить и устранить возможные несоответствия и обеспечить одинаковый уровень функциональности и эстетики. Такой системный подход помогает создать интуитивно понятный и привлекательный дизайн, который не требует отдельной разработки для каждого типа устройства.

Ещё JavaScript значительно влияет на скорость и отзывчивость сайтов. Одним из ключевых факторов является время загрузки страницы. Чем больше и тяжелее скрипты, тем дольше идет их скачивание и распаковка браузером перед тем, как страница полностью откроется. Поэтому важно сокращать объем файлов, которая удаляет все лишние пробелы, комментарии и сокращает

имена переменных, а также разделять крупные скрипты на меньшие части, которые можно загружать по мере необходимости.

Влияние JavaScript заметно и на работу со структурой страницы. Частые и неправильные изменения DOM вызывают операции перерасчета размеров и перерисовки элементов. Каждое такое изменение тормозит работу интерфейса и вызывает задержки. Поэтому важно использовать методы группировки изменений и обновлять DOM за один раз, а не по частям.

Все эти аспекты в совокупности обеспечивают более быструю и отзывчивую работу страницы. Неправильное или неаккуратное использование JavaScript быстро превращает сайт в тормозную машину, потому что создает лишнюю нагрузку на устройство пользователя, вызывает задержки и ухудшает пользовательский опыт. Поэтому очень важно оптимизировать скрипты, следить за их размером и порядком выполнения, чтобы сайт работал как часы, а пользователи оставались довольны быстрым и плавным взаимодействием.

1.2 Серверная часть

Для реализации серверной части хорошо подходит PHP. Это мощный скриптовый язык, специально предназначенный для разработки серверных приложений, который широко применяется в веб-разработке по всему миру. Он заслуженно считается одним из самых популярных инструментов для создания динамических сайтов и веб-сервисов, благодаря своей простоте и доступности, а также богатому набору возможностей, необходимых для реализации самых разнообразных задач. Кроме того, благодаря активному сообществу и богатству готовых решений и библиотек, работа с ним становится более удобной и быстрой, что позволяет быстро разрабатывать и внедрять серверные компоненты.

Одним из ключевых преимуществ PHP является его простота освоения. Он обладает интуитивно понятным синтаксисом, что позволяет начинающим разработчикам быстро войти в курс дела и начать создавать полноценные веб-

приложения без необходимости долгого обучения. Это делает его идеальным выбором для тех, кто только начинает свой путь в веб-программировании.

Тем не менее, несмотря на свою простоту, PHP обладает мощным арсеналом средств и возможностей, которые позволяют создавать сложные, масштабируемые и интерактивные сайты. Он отлично подходит для быстрого генерирования HTML-страниц, вставляя актуальные данные из баз данных или результатов выполнения скриптов. Такой подход обеспечивает динамическое обновление содержимого сайта, что делает его более актуальным и привлекательным для посетителей.

Помимо этого, PHP отлично взаимодействует с системами управления базами данных, что является одним из его основных преимуществ. Особенно популярной системой хранения данных считается MySQL — бесплатная, надежная и легко интегрируемая с PHP. Веб-приложения, построенные на PHP и MySQL, способны эффективно обрабатывать большие объемы информации, обеспечивая быструю работу и высокую отзывчивость интерфейса. Это особенно важно для сайтов с большим количеством пользователей, предполагающими постоянные обновления и обращения к базе данных.

Интеграция PHP с базами данных открывает широкие возможности для реализации различных функциональных особенностей, что помогают обеспечить удобство пользования сайтом для конечных пользователей, повысить его эффективность и расширить потенциальные возможности бизнеса.

Кроме взаимодействия с базами данных, PHP широко используется для обработки форм и взаимодействия с пользователями. Скрипты на PHP позволяют реализовать процесс регистрации новых пользователей, входа в личный кабинет и многое другое. При этом, ключевым аспектом является правильная проверка данных. PHP предоставляет встроенные механизмы для валидации и фильтрации пользовательского ввода, что помогает защитить сайт от вредоносных атак, таких как SQL-инъекции или XSS-атак. Защита личных данных, а также информации о платежах, является приоритетом для любого

сайта с коммерческим уклоном, и PHP предлагает множество инструментов для обеспечения безопасности данных.

Обеспечение безопасности данных становится особенно актуальным, когда речь идет о работе с пользовательскими аккаунтами, платежной информацией или конфиденциальными данными. PHP управляет сессиями, системой авторизации и аутентификации, что позволяет надежно хранить и проверять идентификаторы входа пользователя. Благодаря этому обеспечивается защита от несанкционированного доступа и повышается уровень доверия к проекту. А также, в случае возникновения угроз или неправомерных действий, PHP предоставляет широкие возможности для мониторинга и устранения уязвимостей.

Его функционал не ограничивается только обработкой данных и взаимодействием с базами данных. Это универсальный язык, который работает практически на всех популярных операционных системах. Благодаря этому, разработчики и системные администраторы получают большую гибкость в выборе серверной платформы, а также облегчается развертывание и масштабирование проектов. Например, сайт, созданный на PHP, может легко перемещаться между различными хостинг-провайдерами без необходимости переписывать код или делать трудоемкие настройки. Это значительно упрощает поддержку и развитие проекта, снижая затраты времени и ресурсов.

Помимо этого, PHP является языком с открытым исходным кодом, что означает отсутствие лицензионных затрат и возможность модифицировать его под свои нужды. Активное сообщество разработчиков по всему миру постоянно создает и обновляет библиотеки, фреймворки и расширения. Эти инструменты позволяют ускорить процесс разработки, повысить качество кода и внедрять современные технологии

Для взаимодействия с базой данных в PHP существует класс PDO. Это современный и мощный интерфейс, который обеспечивает абстрактный уровень работы с различными системами управления базами данных. Это делает код более универсальным и легко переносимым между разными

системами управления базами данных без необходимости в радикальных изменениях.

Еще одним важным аспектом является диалектный уровень PDO, который помогает писать универсальный код для различных баз данных, что облегчает миграцию или поддержку систем с разными типами системами управления базами данных. Также с помощью PDO реализуются атомарные операции, которые позволяют гарантировать целостность данных при выполнении нескольких связанных запросов, что особенно критично для финансовых приложений или систем с высокой нагрузкой.

Также PDO способствует защите от угроз, таких как SQL-инъекции, за счет использования подготовленных выражений и привязки параметров. Это позволяет разработчикам писать запросы, которые автоматически экранируют вводимые пользователями данные, снижая вероятность уязвимостей. Помимо этого, PDO значительно упрощает обработку ошибок и исключений, то есть в случае возникновения проблем с выполнением запроса или соединением он позволяет удобно ловить исключения и фиксировать ошибки, а также реагировать на них в контролируемом виде. Это особенно важно для повышения надежности и стабильности приложения.

В контексте же безопасности данных, защита от различных угроз — это ключевая составляющая любой современной информационной системы. Особенно, когда речь идет о работе с конфиденциальной информацией, необходимо применять множество методов и технологий, направленных на предотвращение несанкционированного доступа, утечки или повреждения данных. И, безусловно, одним из фундаментальных методов защиты является шифрование.

Это процесс преобразования данных в недоступный для посторонних вид, используя специальные алгоритмы и ключи. Шифрование применяется для защиты передаваемой информации по сети и хранения данных на сервере. Благодаря ему, даже если злоумышленник получит доступ к файлам или трафику, без ключа расшифровать данные будет практически невозможно.

Популярные алгоритмы шифрования обеспечивают безопасность в современных системах.

Помимо этого, существуют два механизма, которые помогают контролировать, кто и что может делать на сайте. Первый из них — аутентификация — процесс подтверждения личности пользователя, который обычно реализуется через ввод с помощью методов идентификации. А второй — авторизация — определяет, какие действия и ресурсы доступны пользователю после успешной аутентификации.

Также важно защитить систему от распространенных веб-угроз. Это включает меры против таких атак, как SQL-инъекции, CSRF и XSS-атак. Каждая из этих угроз требует особых мер защиты для предотвращения взломов системы, например:

- ✓ для защиты от SQL-инъекций используют подготовленные выражения и привязку параметров в запросах;
- ✓ для защиты от CSRF используются специальные токены, чтобы убедиться, что команда исходит именно от авторизованного пользователя;
- ✓ для предотвращения XSS используются функции для удаления или экранирования HTML-тегов.

Для обеспечения надежной защиты системы необходимо учитывать все эти угрозы и применять соответствующие меры. Только комплексный подход к безопасности позволяет минимизировать риски и обеспечить безопасную работу веб-приложения.

1.3 База данных

Для выполнения запросов существует язык SQL, ведь это структурированный язык запросов, который является одним из самых важных инструментов при работе с базами данных. Он предназначен для управления и организации данных, позволяя пользователям и разработчикам легко и эффективно взаимодействовать с информацией, которая хранится в системах управления базами данных. По сути, SQL служит языком коммуникации

между пользователем и базой данных, и с его помощью можно выполнять самые разные операции.

Начинается всё с создания таблиц — это первый и важный шаг в построении базы данных. При создании таблицы задается её название и структура, то есть поля и их типы данных. Это помогает организовать хранение информации в понятном и структурированном виде, что значительно облегчает доступ и управление данными.

Впоследствии, когда таблица становится ненужной или требует изменений, её можно удалить. Этот процесс полностью очищает таблицу и все связанные с ней данные, поэтому к этому нужно подходить с осторожностью.

При создании таблицы очень важно заранее определить её структуру — какие именно поля она будет содержать. Для каждого поля выбирается подходящий тип данных, будь то числовой, текстовый или дата. Это повышает эффективность работы базы данных, обеспечивает целостность данных и помогает избежать ошибок при вводе. Такая структурированная организация служит шаблоном, по которому хранится вся информация, что в дальнейшем облегчает быстрое и точное получение нужных данных.

Со временем требования к базе данных могут измениться, и тогда потребуется её изменение. Таблицы можно дополнять новыми полями, удалять ненужные или изменять существующие. Также возможна корректировка типа данных в колонке или настройка связей между таблицами, что помогает поддерживать актуальность и согласованность данных. Гибкое управление структурой базы данных позволяет адаптировать систему под новые задачи без необходимости полностью её перестраивать.

SQL позволяет добавлять новые записи, обновлять существующие или удалять устаревшие данные. Эти операции помогают поддерживать актуальную и точную информацию, а также обеспечить гибкое управление данными. Чтобы получить из базы данных нужную информацию, используют сложные запросы с условиями фильтрации, сортировками и группировками.

Такие инструменты делают анализ информации более точным и помогают принимать обоснованные бизнес-решения.

Иногда данные хранятся в нескольких таблицах, и для полноценного анализа необходимо объединять эти данные. SQL предлагает мощные возможности для соединения таблиц, что позволяет просматривать связанные сведения вместе, например, заказы клиентов с деталями или связи между товарами и категориями.

Все эти возможности делают SQL универсальным и популярным языком, без которого трудно представить современную работу с данными и информационными системами. Его изучение и практика помогают создавать надежные, быстрые и масштабируемые базы данных — важнейший аспект эффективного функционирования различных приложений и сервисов в любой сфере деятельности.

Однако, работая только с одним SQL, сложно реализовать весь потенциал управления базами данных, поэтому существуют системы, основанные на этом языке, которые помогают расширить его возможности. Одним из самых популярных решений является MySQL — мощная и широко используемая платформа, которая воплощает все перечисленные возможности и дополняет их своими особенностями. Это связано с тем, что он отлично подходит для обработки больших объемов информации и обеспечивает стабильную работу, её логотип представлен на рисунке 1.



Рисунок 1. Логотип MySQL

Также она бесплатна и открыта, значит, любой желающий может скачивать, использовать и модифицировать её под свои нужды. Во-вторых,

MySQL работает быстро и стабильно, даже при больших объемах данных и большом числе пользователей. В-третьих, она легко масштабируется, что важно для растущих проектов и крупных систем. Всё это делает MySQL отличным решением для сайта-визитки, интернет-магазина, системы учета или любой другой программы, где необходимо много информации и требуется быстрая обработка запросов.

Вдобавок, благодаря своему широкому распространению и большой пользовательской базе, существует множество ресурсов, учебников и сообществ, которые помогают новичкам и профессионалам решать любые возникающие вопросы.

Так как MySQL поддерживает множество платформ и операционных систем, её можно интегрировать практически в любой IT-инфраструктуре. Не менее важно то, что инструмент регулярно обновляется и дорабатывается разработчиками, что повышает безопасность и расширяет функциональные возможности системы.

Одним из главных преимуществ MySQL является то, что она использует язык структурированных запросов — SQL. Этот язык позволяет писать команды для создания, изменения и получения данных из базы. В MySQL есть множество функций и команд, которые помогают автоматизировать работу с информацией. Она очень гибкая и насыщенная возможностями, такими как:

- ✓ создание таблицы, указывая тип данных для каждого поля;
- ✓ добавление новых полей или удаление ненужных;
- ✓ установка связи между таблицами для иерархии данных;
- ✓ фильтрация данных по различным условиям;
- ✓ группировка и сортировка информации для анализа;
- ✓ формирование отчёта и сводки.

Всё это делает работу с базами данных очень удобной, позволяя автоматизировать многие рутинные задачи.

Помимо перечисленных возможностей, MySQL постоянно развивается и внедряет новые функции, что делает её одним из самых актуальных решений

в области управления базами данных. Например, современная версия включает инструменты для работы с JSON-данными, что позволяет хранить и обрабатывать неструктурированные данные прямо внутри базы. Это значительно расширяет спектр применений системы, делая её подходящей для современных веб-приложений, где часто используются гибкие форматы хранения информации.

Также MySQL ориентирована на безопасность. В ней реализованы механизмы аутентификации, шифрования данных и управления правами доступа, что позволяет обеспечивать конфиденциальность и защиту информации при работе с важными или личными данными. Встроенные функции резервного копирования и восстановления позволяют при необходимости быстро восстановить базу данных без потери информации.

Кроме того, MySQL обладает высокой степенью совместимости с различными языками программирования, такими как PHP, Python, Java и другими. Это облегчает интеграцию базы данных в любые программные решения и позволяет создавать полноценно функционирующие веб-сервисы, мобильные приложения и корпоративные системы. Не стоит забывать и о том, что для оптимизации работы с большими базами данных часто используют мощные средства кеширования и индексации, что существенно ускоряет выполнение сложных запросов.

1.4 Программные средства для разработки

Для удобства в управлении базой данных через браузер существует популярное веб-приложение phpMyAdmin, которое предоставляет пользователям интерфейс для работы с базами данных MySQL прямо через браузер. Его основная задача — сделать работу с базой данных максимально простой и доступной, особенно для тех, кто не обладает глубокими знаниями SQL или предпочитает графический интерфейс, а не командную строку.

Его основные преимущества включают регулярные обновления, которые делают его безопасным и современным инструментом для защиты данных. Он

доступен бесплатно и легко устанавливается на любой сервер с поддержкой PHP, работает через браузер, что исключает необходимость установки сложных программ или навыков работы с командной строкой. Его логотип представлен на рисунке 2.



Рисунок 2. Логотип phpMyAdmin

Это делает его идеально подходящим для начинающих, студентов, фрилансеров, маленьких проектов и небольших команд. Благодаря простому интерфейсу он позволяет максимально упростить выполнение многих рутинных задач, освобождая время для разработки или анализа данных.

Он предоставляет очень удобный и интуитивно понятный визуальный интерфейс, который делает работу с программой более простой и приятной, позволяя быстро и без лишних усилий выполнять действия, например:

- ✓ вместо команд SQL или командной строки, можно работать через формы и кнопки;
- ✓ можно легко создавать, изменять и удалять таблицы, добавлять новые поля и редактировать их с помощью графических элементов;
- ✓ визуально просматривать структуру базы данных, связи между таблицами;
- ✓ фильтровать и сортировать данные по любым условиям;
- ✓ настраивать права доступа и управлять аккаунтами пользователей базы данных.

Популярность phpMyAdmin объясняется и тем, что он отлично адаптируется под нужды пользователей разного уровня — от новичков,

начинающих знакомство с базами данных, до профессиональных разработчиков и системных администраторов. В его интерфейсе легко реализовать настройку прав доступа для разных пользователей, а также управлять их аккаунтами, что важно для защиты данных. В дополнение к этому, регулярные обновления не только добавляют новые функции, но и повышают уровень безопасности, что особенно актуально при работе с важной информацией.

Для удобного локального развертывания и тестирования веб-приложений существует Open Server Panel. Это мощный, бесплатный и портативный инструмент для локальной разработки веб-сайтов и приложений. Он предназначен для веб-разработчиков, студентов, тестировщиков и всех, кто работает с веб-технологиями и нуждается в быстром, надежном и удобном локальном сервере.

Open Server включает в себя популярные веб-серверы — Apache и Nginx, что позволяет запускать сайты и тестировать их работу в условиях, приближенных к реальному серверу. Можно выбрать, какой сервер использовать, в зависимости от требований проекта, он поддерживает:

- ✓ версии PHP от 5.6 до 8.4, что позволяет тестировать проекты на разных версиях интерпретатора;
- ✓ базы данных для хранения информации о сайте или приложении;
- ✓ возможность подключаться к проектам через FTP для тестирования.

Интуитивно понятный интерфейс программы позволяет легко добавлять новые проекты — создавая виртуальные хосты простым кликом, управлять доменными именами внутри локальной сети и настраивать пути к папкам проектов. В дополнение, программа автоматически настраивает конфигурационные файлы веб-сервера, что значительно упрощает работу.

Кроме того, Open Server поддерживает работу с различными системами контроля версий, что позволяет вести удобную работу с изменениями и совместную разработку. Встроенная поддержка различных веб-фреймворков,

что делает его отличным решением для быстрого развертывания тестовых сред или проверки функционирования различных решений. Для повышения удобства работы, программа позволяет делать снимки состояния серверной среды и возвращаться к ним при необходимости, что особенно полезно при обновлениях или экспериментальных настройках. Также стоит отметить наличие расширенных настроек безопасности, позволяющих контролировать доступ и защищать локальную среду разработки.

Удобная панель управления обеспечивает запуск и остановку серверных служб, просмотр логов ошибок и доступа, а также быстрый доступ к важным функциям, таким как перезапуск, остановка и настройка, обеспечивая максимально комфортное и эффективное управление всеми аспектами работы сервера. Open Server поддерживает автоматическую установку популярных систем управления сайтами, а также различные фреймворки и движки.

Для управления версиями проекта, отслеживания изменений и совместной работы над кодом отлично подходит Git. Это распределённая система контроля версий, которая активно используется в сфере разработки программного обеспечения для отслеживания изменений в коде, управления различными версиями проекта и обеспечения командной работы без риска потери важной информации, его логотип представлен на рисунке 3.



Рисунок 3. Логотип Git

В отличие от централизованных систем контроля версий, он позволяет каждому разработчику иметь полноценную копию всего репозитория на своём компьютере, что обеспечивает большую независимость и безопасность данных. Это означает, что можно работать офлайн, делать коммиты,

просматривать историю изменений, создавать новые ветки и тестировать их без необходимости постоянного подключения к серверу. Весь прогресс сохраняется в локальных копиях, а при необходимости изменения можно синхронизировать с общего репозитория, обмениваться ими с командой или публиковать на удалённых платформах. Среди его ключевых преимуществ является скорость работы: операции такие как коммиты, просмотр истории или создание веток выполняются очень быстро, что существенно повышает эффективность разработки.

Также, благодаря мощной системе ветвления и слияния, разработчики могут одновременно работать над различными задачами, создавать новые ветки для разработки новых функций, исправлений ошибок или экспериментов, а затем без лишних трудностей объединять эти ветки с основной. Это способствует более структурированной и управляемой работе, а также позволяет тестировать новые идеи, не мешая основной линии разработки.

Все коммиты фиксируют состояние проекта в конкретный момент времени, что позволяет любому участнику команды иметь полное представление о прогрессе и легко возвращаться к старым версиям в случае необходимости. Это похоже на систему резервных копий, только встроенную прямо в процесс разработки. Такой подход помогает избежать потери данных при сбое или ошибке, а также облегчает внедрение новых технологий, исправление ошибок и тестирование новых решений.

2. РАЗРАБОТКА ИНТЕРНЕТ-МАГАЗИНА

Создание современного интернет-магазина — это сложный и многоэтапный процесс, который включает в себя разработку эффективной архитектурной системы, реализацию пользовательских интерфейсов и обеспечение стабильной работы всех функций сайта. Важно учитывать не только технические аспекты, такие как безопасность данных, производительность и масштабируемость, но и удобство для конечного пользователя.

2.1 Централизованные системные модули

В проекте интернет-магазина используются отдельные файлы для подключения к базе данных, а также для формирования универсальных элементов сайта. Эти файлы подключаются на каждой странице сайта с помощью конструкции, представленной ниже:

```
include_once __DIR__ . "/config/config.php";
```

Данная конструкция обеспечивает несколько важных преимуществ, облегчающих разработку, а именно она:

- ✓ гарантирует централизованный доступ к ключевым ресурсам и настройкам, что облегчает управление конфигурацией сайта;
- ✓ способствует поддержанию единого стиля и оформления сайта, так как все страницы используют одинаковые шаблоны и компоненты;
- ✓ обеспечивает безопасность за счёт предотвращения повторного подключения одних и тех же файлов;
- ✓ упрощает процесс поддержки и доработки проекта, так как изменения внутри подключаемого файла автоматически применяются ко всем страницам сайта.

Первым важным и централизованным модулем является файл конфигурации, предназначенный для подключения к базе данных. Этот файл играет ключевую роль в обеспечении бесперебойной работы интернет-

магазина, поскольку от правильной настройки зависит возможность корректного взаимодействия всей системы с хранилищем данных. В базе данных хранится вся ключевая информация: данные о товарах, учетные записи пользователей, заказы, статусы доставки, а также прочие необходимые сведения, обеспечивающие функционирование магазина.

Для подключения к базе данных используется конфигурационный файл, в котором задаются параметры подключения: сервер базы данных, имя базы, пользователь и пароль. Правильная настройка этого файла обеспечивает безопасность и эффективность работы системы, а также облегчает последующие обновления и техническое обслуживание.

Ниже приведён пример кода, реализующего подключение к базе данных с использованием расширения PDO, которое рекомендуется для работы с базами данных благодаря своей безопасности и гибкости:

```
$host = "MySQL-8.4";
$dbname = "store";
$username = "root";
$password = "";
try {
    $link=newPDO("mysql:host=$host;dbname=$dbname",
$username, $password);
} catch (Throwable $error) {
    echo "Ошибка подключения к базе данных. Код
ошибки: " . $error->getCode();
    exit;
}
```

После файла конфигурации ключевым компонентом является модуль динамического отображения элементов шапки сайта. В интернет-магазине реализована система, которая изменяет содержимое и функциональность верхней части страницы в зависимости роли текущего пользователя. Такой подход значительно повышает удобство пользования сайтом, упрощая навигацию и повышая безопасность, а также делая интерфейс более интуитивно понятным для разных категорий посетителей. Это позволяет, например, показывать только те разделы и кнопки, которые актуальны для конкретного пользователя. Благодаря такому механизму пользователь

получает более персонализированный опыт, а администратор — возможность более гибко управлять доступом и функционалом сайта в зависимости от ролей пользователей:

- ✓ гость — не авторизованный пользователь, для которого отображаются вкладки входа или регистрации;
- ✓ зарегистрированный пользователь — имеет доступ к профилю, корзине и техподдержке;
- ✓ администратор — видит расширенное меню для управления настройками сайта;
- ✓ курьер — имеет доступ к информации о доставках заказов;
- ✓ специалист техподдержки — видит информацию о вопросах пользователей.

Такая реализация достигается благодаря централизованной проверке ролей пользователя, после чего происходит динамическая генерация соответствующих элементов интерфейса. Ниже представлен пример кода, реализующего такую функциональность:

```
if (isUserAuth()) {  
    if (isAdmin()) $header .= "<a  
href='/admin/admin.php'></a>";  
    if (isDeliver()) $header .= "<a href = '/deliver/  
allOrders.php'> </a> <a  
href='/deliver/myOrders.php'></a>";  
    if (isSupport()) $header .= " <a href = '/support/  
allSupport.php'></a> <a href='/user/support.php'>  
</a>";  
    else $header .= "<a href='/user/support.php'></a>";  
    $header .= "<a href='/user/basket.php'> </a> <a  
href='/user/favorites.php'> </a> <a href=  
'/user/profile.php'></a>";  
} else $header .= "<span><a href='/user/auth.php'>  
</a> <a href='/user/reg.php'></a></span>";
```

Персонализированное отображение позволяет каждому пользователю видеть только те ссылки и функциональные элементы, которые действительно ему необходимы. При совмещении ролей — например, если администратор

также выполняет функции курьера — пользователю отображаются все соответствующие разделы. Это уменьшает избыточность информации и делает взаимодействие с интерфейсом максимально удобным.

А для еще большего повышения удобства и эффективности работы сайта на страницах реализован автоматический механизм подключения ресурсов — скриптов и стилей — соответствующих текущей странице. Такой подход позволяет не только сократить ручную настройку интерфейса, но и обеспечить динамическую загрузку необходимых элементов, делая работу с проектом более модульной и структурированной.

Идея заключается в том, что при загрузке каждой PHP-страницы система автоматически определяет её имя и проверяет наличие соответствующих файлов со скриптами и стилями в специальных папках. Если такие файлы существуют, они подключаются к документу, обеспечивая необходимую функциональность и оформление страницы. Ниже представлен пример кода, реализующего данный механизм:

```
$fileName=pathinfo($_SERVER["SCRIPT_NAME"],PATHINFO_
_FILENAME);
$links = "";
if (file_exists(__DIR__."/js/$fileName.js")){
    $links .= "<script src='/js/$fileName.js'
defer></script>";
}
if (file_exists(__DIR__."/css/$fileName.css")){
    $links .= "<link rel='stylesheet'
href='/css/$fileName.css'>";
}
```

В результате на каждой странице автоматически подключаются те ресурсы, которые имеют совпадающее имя с именем PHP-файла. Например, для страницы profile.php будут подключены файлы profile.js и profile.css, если они существуют в специальных папках. Такой подход делает процесс управления ресурсами более удобным и автоматизированным, что значительно упрощает добавление новых страниц, а также способствует более чистой и организованной структуре проекта.

В этом контексте важной роль играет и встроенный механизм динамического отображения элементов, который делает интерфейс более адаптивным и персонализированным для каждого пользователя, а футер сайта становится настоящим инструментом вовлечения и взаимодействия в случайных и интересных форматах. Нижняя часть страницы становится не только информационной, но и функциональной, что способствует улучшению пользовательского опыта. Основные элементы футера включают в себя две кнопки, каждая из которых выполняет особую роль.

Первая кнопка — ссылка на страницу произвольного товара, выбранного случайным образом из базы данных. Это реализуется следующим образом:

```
$randomItem = makeSelectQuery("SELECT `id_items`  
FROM `items` ORDER BY RAND() LIMIT 1", [], true);  
<a href = "/user/aboutItem.php?id_item = <?=  
$randomItem["id_items"] ?? 1 ?>" class="footer-item  
button"> </a>
```

Если по какой-то причине не удастся получить случайный товар, то автоматически подставляется значение по умолчанию, чтобы ссылка оставалась рабочей. Это делает взаимодействие с сайтом более динамичным и интерактивным, пробуждая интерес пользователя к новым товарам без необходимости искать их вручную.

Вторая кнопка позволяет пользователю перейти в случайную категорию товаров. Ниже представлена реализация данного механизма:

```
$randomCategory = makeSelectQuery("SELECT  
`id_items_type` FROM `items_type` ORDER BY RAND() LIMIT  
1", [], true);  
<a href = "/user/index.php?items_type_id_items = <?=  
$randomCategory["id_items_type"] ?? 1 ?>" class="footer-  
item button"> </a>
```

Эта возможность помогает посетителям случайным образом исследовать разные разделы каталога, открывая для себя новые товары и категории, что делает процесс покупки менее монотонным и более увлекательным. Такие небольшие интерактивные элементы позволяют посетителю почувствовать,

что сайт живёт и постоянно предлагает что-то новое, стимулируя его возвращаться снова и снова.

В рамках реализации этого функционала создан специальный механизм для безопасной и удобной загрузки изображений с сервера по указанным путям. Этот механизм предназначен для обеспечения быстрого и защищённого отображения картинок, таких как аватары, изображения товаров и элементы поддержки, без необходимости напрямую обращаться к файлам через URL.

Главная идея заключается в том, что сервер получает запрос с GET-параметром, который указывает путь к изображению внутри определённой директории, например, аватары, товары или поддержка. Далее скрипт проверяет, что указанный путь соответствует допустимым папкам, чтобы избежать доступа к запрещённым областям системы. Также выполняется проверка на наличие опасных элементов, которые могут использоваться для выхода за пределы разрешённой директории, что предотвращает атаку обхода каталога. Ниже представлен пример кода, реализующего данный механизм:

```
if (!empty($_GET["path"])) {
    if (!in_array($dir, ["avatars", "items",
"supports"])) || strpos($path, "..") != false)
http_response_code(403);
    if ($dir == "supports") {
        $check = makeSelectQuery("SELECT
`users_id_talks`, `users_support_talks` FROM `supports`
JOIN `talks` ON `id_talks` = `talks_id_supports` WHERE
`image_supports` = ?", [$file], true);
        if (!in_array(getUserID(), $check))
http_response_code(403);
    }
    $mime = mime_content_type($fullPath);
    header("Content-Type: $mime");
    readfile($fullPath);
}
```

Если проверка прошла успешно, скрипт ищет файл по указанному пути. В случае отсутствия файла на сервере, подаётся изображение по умолчанию, которое служит резервным изображением на случай отсутствия или повреждения файла. В противном случае сервер считывает тип содержимого

файла и возвращает его клиенту с правильным MIME-типом, что гарантирует корректное отображение картинки в браузере. Особое внимание уделяется безопасности: перед выводом изображения проверяется принадлежность пользователя к спискам поддержки, чтобы не допустить несанкционированный доступ к приватным изображениям поддержки. В случае несоответствия пользователь получает ошибку 403, а некорректные или опасные запросы — ошибку 406.

Этот подход обеспечивает безопасную и стабильную загрузку изображений, способствует защите данных и облегчает поддержку сайта, поскольку все картинки хранятся централизованно и доступны только через проверенные пути. Такой механизм повышает устойчивость сайта к ошибкам и злоумышленникам, и одновременно делает работу с изображениями более удобной и структурированной.

В результате, благодаря этим небольшим, но приятным и функциональным деталям, пользовательский опыт превращается в увлекательное путешествие, где каждый заход — это шанс открыть что-то неожиданное. Также это создает атмосферу дружелюбия и заботы, ведь сайт становится не только инструментом покупки, но и средой для исследования, вдохновения и приятных сюрпризов.

2.2 Процесс регистрации и авторизации пользователей

Регистрация новых пользователей в интернет-магазине реализована с учетом баланса удобства и безопасности. Для обеспечения простоты процесса регистрации, система проверяет, авторизован ли уже текущий пользователь. Если пользователь уже вошел в систему, повторная регистрация становится невозможной — происходит автоматическое перенаправление на главную страницу сайта. Пример реализации этой проверки:

```
if (isUserAuth()) {  
    redirect();  
}
```

Это предотвращает создание дубликатов аккаунтов и обеспечивает целостность данных с помощью функция `isUserAuth`. Она возвращает истину, если у пользователя есть активный идентификатор в системе, и ложь — если пользователь не авторизован. Основная идея состоит в том, чтобы проверить наличие текущего ID пользователя через функцию `getUserID`, которая реализована так:

```
$result = null;
if (!empty($_SESSION["id_user"])) {
    $result = $_SESSION["id_user"];
}
return $result;
```

Данная функция использует глобальную переменную `$_SESSION`, где хранится ID текущего пользователя под определённым ключом. Если пользователь авторизован, эта переменная содержит его уникальный идентификатор, иначе ничего. Это позволяет системе быстро и эффективно определить статус авторизации пользователя без необходимости обращения к базе данных каждый раз при проверке. Кроме того, использование сессий обеспечивает сохранность информации о входе пользователя в течение всей сессии, что делает проверку более стабильной и безопасной.

Затем, когда пользователь заполняет и отправляет регистрационную форму, вся информация немедленно поступает на сервер, где автоматически проходит обработку и строгую валидацию через специальную функцию `getValidatedData`.

Данная функция отвечает за комплексную валидацию массива данных, чтобы убедиться, что все поля заполнены верно и соответствуют заранее определённым правилам. Функция начинает работу с подготовки стандартной структура с тремя ключами:

- ✓ `data` для очищенных данных;
- ✓ `errorField` для хранения информации о наличии ошибок по каждому полю;
- ✓ `isCorrect` для отметки корректности всей проверки.

Если на вход функции поступает пустой массив данных, она сразу возвращает базовый результат, что предотвращает лишнюю обработку и ускоряет выполнение скрипта.

Далее, для текущего типа данных подгружаются правила валидации через вызов функции `getValidationRules`, которая реализована следующим образом:

```
$result = [];  
if ($file == "") $file =  
basename($_SERVER["SCRIPT_NAME"]);  
foreach ($rules as $key => $rule) {  
    if (in_array($file, $rule["files"])) {  
        $result[$key] = $rule;  
    }  
}  
return $result;
```

Помимо этого, внутри функции хранится массив правил для каждого поля формы, что позволяет гибко управлять процессом валидации и обработки данных. В каждом правиле содержится набор пунктов, отвечающих за разные аспекты обработки, что обеспечивает модульный и удобный механизм настройки валидации:

- ✓ `files` указывает, в каких скриптах или частях сайта применяется данное поле;
- ✓ `required` отвечает за обязательность, если поле является обязательным, то оно должно быть заполнено, иначе валидация не будет успешной;
- ✓ `canUpdate` определяет, можно ли изменять это поле после того, как оно было добавлено в базу данных;
- ✓ `returnedValue` обозначает, нужно ли возвращать обработанное значение после проверки паттерном;
- ✓ `pattern` содержит функцию-проверку, которая задаёт допустимый формат для значения.

Без этих правил функция мгновенно завершает выполнение, что позволяет экономить ресурсы и избегать лишних проверок. Затем в цикле

перебирается каждое поле формы. На каждом шаге сначала проверяется, существует ли правило для данного ключа — если его нет, это фиксируется, и выполнение продолжается дальше, чтобы не прерывать обработку из-за отсутствия правила. Если значение поля не пустое, и у правила указано, что после проверки должно быть возвращаемое значение, отличное от изначального, вызывается соответствующая паттерн-функция для проверки и возможной обработки значения. Данная логика реализована следующим образом:

```
if ($value != "" && $rule["returned_value"]) {  
    $returnedValue = $rule["pattern"]($value);  
    if ($returnedValue !== false) {  
        $result["errorField"][$key] = 0;  
        $result["data"][$key] = $returnedValue;  
    } else $result["errorField"][$key] = 1;  
}
```

Если все проверки прошли успешно, результат считается успешным, и в итоговой структуре для соответствующего поля отмечается отсутствие ошибки, то есть устанавливается флаг, что поле прошло проверку без особых проблем. Специальная обработка предусмотрена для паролей, если в массиве данных присутствует подтверждение пароля, и оно не совпадает с основным паролем, в этом поле фиксируется ошибка, чтобы пользователь смог исправить несоответствие. В случае, когда значение не прошло проверку паттерном, для этого поля также фиксируется ошибка, что помогает точно определить проблемное место и обеспечить корректность данных перед сохранением или дальнейшей обработкой.

Если поле не является обязательным, и оно оказалось пустым, то для него также фиксируется, что ошибок нет, и увеличивается счетчик корректных полей, что позволяет системе корректно учитывать такие случаи. Для всех остальных полей, которые не подошли по предыдущим условиям, запускается паттерн-проверка, если она не прошла, то в результатах фиксируется ошибка в этом поле. Если же проверка прошла успешно, поле считается валидным, и ошибок для него не возникает. Аналогично, для обязательных полей,

отсутствие значения или неудачная валидация также засчитываются как ошибка, что предотвращает сохранение некорректных данных. Пример реализации этой логики представлен ниже:

```
$value = trim($value);
$result["data"][$key] = $value;
if (!$rule["required"] && $value == "") {
    $currentCountCorrect++;
    $result["errorField"][$key] = 0;
    continue;
}
$result["errorField"][$key] = !$rule["pattern"]($value) ? 1 : 0;
```

После завершения основных проверок осуществляется еще один цикл, в котором проверяется наличие обязательных полей в переданных данных. Для каждого правила проверяется, входит ли соответствующее поле в массив данных. Если поле отсутствует, а оно обязательно и при этом не допускает обновление, для такого поля фиксируется ошибка. В результате общая оценка валидности системы считается положительной только при выполнении обоих условий, если все проверки прошли успешно и обязательные поля заполнены.

Вся важная информация о результатах собирается в один массив. Этот массив используется далее для решения, следует ли принимать и сохранять полученные данные или просить пользователя исправить ошибки. Такой подход помогает исключить из обработки неправильные данные, неаккуратные адреса электронной почты, пустые обязательные поля и другие ошибки, способные снизить качество данных. Ниже представлен пример кода, реализующего данный механизм:

```
$hasAllRule = true;
foreach ($validationRules as $key => $rule) {
    if (!key_exists($key, $array) &&
        $rule["required"] && !$rule["canUpdate"]) {
        $result["errorField"][$key] = 1;
        $hasAllRule = false;
    }
}
$result["isCorrect"] = $currentCountCorrect ==
count($array) && $hasAllRule;
```

После завершения процесса проверки введенная пользователем информация сохраняется в сессии. Это делается для того, чтобы в случае ошибок форма могла быть автоматически заполнена данными, которые пользователь уже ввел, и ему не пришлось вводить все заново. Особое внимание уделяется сравнению двух полей с паролями, если они не совпадают, пользователю отправляется соответствующее сообщение, и он может исправить ошибку, не вводя все остальные данные заново.

Также обязательной проверке подлежит поле с электронной почтой. Система ищет, есть ли в базе данных пользователь с таким адресом. Если совпадение найдено, пользователю сразу выводится сообщение о том, что такая почта уже зарегистрирована, и регистрация не продолжается. Если адрес уникален и его нет в базе, система разрешает продолжить регистрацию.

И когда все проверки успешно завершены, система формирует SQL-запрос для добавления нового пользователя в базу данных и автоматически добавляет текущую дату регистрации. После успешной регистрации временные данные, использованные для заполнения формы, очищаются, чтобы не мешать следующим операциям. Затем пользователь автоматически перенаправляется на страницу входа в систему, чтобы он мог сразу начать пользоваться своим аккаунтом. Реализация подробно описана в следующем блоке кода:

```
$checkEmail = makeSelectQuery("SELECT `id_users`  
FROM `users` WHERE `email_users` = ?",  
[$validatedData["data"]["email_users"]], true);  
if (empty($checkEmail)) {  
    $result = getInsertSQL($validatedData["data"]);  
    $isSuccess = makeSafeQuery("INSERT INTO `users`  
($result[sql]) VALUES ($result[question])",  
$result["params"]);  
    if ($isSuccess) redirect("user/auth.php");  
}
```

Если же регистрация не прошла успешно по каким-либо причинам, все ошибки и неточности фиксируются и сохраняются в сессии. Эти сообщения затем отображаются в интерфейсе, чтобы пользователь четко понял, какая

именно часть данных была введена неправильно и что нужно исправить. Такой подход помогает снизить возможные трудности при регистрации и значительно повышает удобство использования сайта для пользователя.

Механизм авторизации пользователей работает по принципу, во многом схожему с регистрацией, но при этом сосредоточен на проверке существующего аккаунта и обеспечении безопасности входа. Сначала система проверяет с помощью функции `isUserAuth`, не вошёл ли пользователь уже в систему, если он уже авторизован, его сразу перенаправляют на главную страницу, чтобы избежать повторного входа и обеспечить правильную работу сессий.

При отправке формы, полученные данные фиксируются и валидируются с помощью функции `getValidatedData`, которая фильтрует и проверяет поля по строгим правилам, а результаты этой проверки сохраняются в сессии. Это позволяет при ошибках в форме повторно заполнить поля и вывести сообщения об ошибках под соответствующими полями, что делает процесс удобнее для пользователя. Для реализации данной функции используется следующий код:

```
$validatedData = getValidatedData($_POST);  
$_SESSION["data"] = $validatedData["data"];
```

Когда данные успешно проходят проверку, система выполняет запрос к базе данных, чтобы найти пользователя по введённой почте. Если по указанной почте пользователь не найден, отображается сообщение, что такой пользователь отсутствует. Это поможет предотвратить попытки входа с несуществующими аккаунтами и повысит безопасность системы.

Если же пользователь найден, система сравнивает введённый пароль с хешем, сохранённым в базе данных, с помощью функции `password_verify`. Если пароли не совпадают, пользователю выводится сообщение о неправильном пароле, и попытка входа прекращается.

После успешного совпадения пароля происходит дополнительная проверка, находится ли аккаунт в блокировке. Если аккаунт заблокирован,

система выводит соответствующее сообщение, и вход не допускается, что помогает предотвратить несанкционированный доступ в случае блокировки пользователя по каким-либо причинам.

Если же аккаунт не заблокирован, а пароль совпадает, в системе создаётся сессия, в которой сохраняется идентификатор пользователя. Это позволяет отслеживать активность пользователя на сайте и сохранять его статус в течение всей сессии. После этого происходит автоматическое перенаправление на главную страницу сайта, чтобы пользователь мог продолжить работу с сервисом. Ниже представлен пример реализации этого механизма на PHP, который учитывает все перечисленные этапы проверки и безопасности:

```
$userInfo = makeSelectQuery("SELECT `id_users`,  
`password_users`, `is_banned_users` FROM `users` WHERE  
`email_users` = ?",  
[$validatedData["data"]["email_users"], true);  
if(!empty($userInfo)&&password_verify($validatedData["data"]["password_users"], $userInfo["password_users"]  
)) {  
    if ($userInfo["is_banned_users"] == 0) {  
        $_SESSION["id_user"]=$userInfo["id_users"];  
        redirect();  
    } else $_SESSION["server"] = "Ваш аккаунт  
заблокирован";  
}
```

Если авторизация по каким-либо причинам не удалась, система автоматически заполняет поле электронной почты значением из сессии, чтобы пользователю не пришлось повторно вводить его заново, что значительно облегчает процесс входа.

Все сообщения об ошибках отображаются прямо под соответствующими полями формы, что делает процесс исправления данных более понятным и быстрым. Таким образом, система авторизации не только придерживается высоких стандартов безопасности, проверяя пароли и блокировки, но и заботится о удобстве и информативности для пользователя на каждом этапе входа, создавая приятный и прозрачный опыт использования.

Переходя к следующему уровню взаимодействия, стоит отметить, что в системе интернет-магазина профиль пользователя реализован как комплексный инструмент самоуправления. Каждый зарегистрированный посетитель получает доступ к своей индивидуальной странице, которая становится центральной точкой управления личной информацией, настройками. Без авторизации доступ к профилю невозможен, что обеспечивает конфиденциальность данных и поддерживает высокий уровень безопасности учетных записей.

Функционал профиля предусматривает возможность редактирования базовых персональных данных. Прежде всего, пользователь видит и может изменить своё имя — эта информация, отображается при оформлении заказов и в истории покупок, а также используется для обращения в службе поддержки.

Обновление данных осуществляется с помощью механизма формирования SQL-запроса через функцию `getUpdateSQL`. Эта функция принимает массив данных, проходится по каждому ключу и значению, и для каждого создает часть SQL-запроса, а также список параметров для подготовки запроса. В результате формируется структура, содержащая строку SQL с подготовленными выражениями и массив параметров, которые затем передаются в базу данных для выполнения обновления, обеспечивая безопасное и динамическое обновление данных в таблице. Вот сама реализация функции:

```
$result = ["params" => [], "sql" => []];
foreach ($array as $key => $value) {
    $result["params"][] = empty($value)?null: $value;
    $result["sql"][] = "`$key` = ?";
}
$result["sql"] = join(", ", $result["sql"]);
return $result;
```

Особую роль в пользовательском интерфейсе играет загрузка и отображение аватара. Наличие личной фотографии не только делает профиль более индивидуальным, но и повышает уровень доверия со стороны службы

поддержки и администрации магазина. Функция мгновенного обновления аватара реализована таким образом, что пользователь может легко заменить или удалить изображение в любой момент. Это достигается с помощью механизма вставки аватара на стороне клиента, ещё до того, как изображение было отправлено на сервер, чтобы пользователь увидел примерный вариант, как будет выглядеть его аватарка. Такой подход делает работу с профилем более удобной и интерактивной.

Также одним из важнейших аспектов на странице профиля является возможность смены пароля. Для этого реализован отдельный блок, где предусмотрены два поля, что уменьшает риск ошибок при смене, гарантируя, что пользователь не случайно введёт неправильный пароль.

Одной из ключевых особенностей системы обновления данных является интеллектуальная отправка информации профиля на сервер. В отличие от стандартных методов, при которых весь набор данных отправляется заново при каждом обновлении, в данном случае отправляются только те данные, которые действительно были изменены пользователем. Такой подход значительно сокращает сетевой трафик и уменьшает нагрузку на сервер, что особенно важно для повышения производительности и скорости работы сайта. Это также позволяет обеспечить более быстрый отклик интерфейса и повысить удобство для пользователя, так как ему не приходится ждать загрузки большого объёма информации. В результате достигается более эффективное использование ресурсов и улучшение общего опыта взаимодействия с системой.

Для реализации этой функциональности на стороне клиента используется специальный скрипт, который привязан к каждой интерактивной форме профиля. Этот скрипт следит за изменениями в полях формы — при любом внесении изменений он автоматически запускает функцию проверки и сравнения текущих значений с исходными, чтобы определить, какие именно данные были изменены.

В каждом вводе или изменении значения в любом поле профиля функция `checkInput` автоматически проверяет данное поле на корректность и сверяет текущее значение с исходным, которое сохраняется при первоначальной загрузке страницы. Если значения совпадают, полю не присваивается атрибут `name`, и оно не включается в состав отправляемых данных при отправке формы. Это позволяет отправлять только изменённые данные, снижая нагрузку на сервер и повышая эффективность процесса обновления

Если же поле было изменено, функция устанавливает специальный атрибут `name` и помечает это поле как требующее отправки на сервер. Таким образом, система точно знает, какие именно поля требуют обновления, что обеспечивает более быструю и точную обработку данных. Основная логика работы реализована на языке JavaScript и представлена ниже:

```
if (rule.check(input) !== false) isCorrect = false;
else isCorrect = true;
rule.isInsertServer[input.id] = isCorrect &&
rule.oldValue[input.id] == rule.currentValue[input.id] ?
1 : 0;
const isUpdate = rule.isInsertServer[input.id] == 0
&& (rule.oldValue[input.id] != input.value ||
input?.files?.length > 0);
const isConnectedInputChange =
rule.connectedInputs[input.id]?.some((input) =>
input.hasAttribute("name"));
if (isCorrect && rule.hasName == true && (isUpdate
|| isConnectedInputChange)) input.setAttribute("name",
rule.nameRule);
else input.removeAttribute("name");
```

Такой механизм позволяет отправлять только действительно изменившиеся данные, а не весь набор полей формы. Это существенно сокращает время обработки запроса сервером и упрощает логику обновления информации в базе данных, поскольку серверу не нужно выполнять лишние операции и проверять дублирующиеся значения. Аналогичная логика применяется ко всем полям в интернет-магазине. Таким образом достигается гибкая и оптимизированная работа с пользовательскими данными в интернет-магазине.

Если же пользователь захочет выйти из своего аккаунта, в интернет-магазине реализована функция выхода из учетной записи, что является важной частью обеспечения безопасности и защиты персональных данных пользователя. Для этого предусмотрена отдельная серверная страница, которая вызывается пользователем по нажатию специальной кнопки на странице профиля.

При обращении к странице выхода происходит отключение индивидуальной сессии пользователя. Система выполняет последовательность действий:

- ✓ удаляет идентификатор пользователя из сессионных данных;
- ✓ очищает информацию о текущей проверенной сессии ;
- ✓ переадресует пользователя на основную страницу сайта.

Это предотвращает возможность несанкционированного доступа к профилю в случае смены владельца устройства, а также исключает отображение персональной информации сторонним лицам. Реализация данного механизма представлена следующим образом:

```
unset($_SESSION["id_user"]);  
clearValidatedSession();  
redirect();
```

После перехода на основную страницу пользователь полностью выходит из своего аккаунта, что подтверждается отсутствием доступа к профилю и другим защищенным разделам до повторной авторизации. Такой подход не только соответствует стандартам веб-безопасности, но и делает работу с персональными данными максимально удобной и защищенной для всех пользователей интернет-магазина.

2.3 Механизм поиска и визуализации каталога товаров

В интернет-магазине основной функционал каталога организован так, чтобы пользователь мог легко искать и фильтровать товары по различным параметрам: названию, цене, категории и другим характеристикам. Для реализации этого на сайте разработана универсальная функция поиска

товаров. Она обрабатывает параметры поиска, получаемые в формате JSON, что позволяет обеспечить гибкую и динамическую фильтрацию ассортимента.

Эта функция позволяет пользователю быстро находить интересующие товары, применять фильтры по различным критериям, таким как популярность, категория, атрибуты и дополнительные параметры. Такой подход делает работу с каталогом максимально удобной и эффективной для пользователя.

Перед выполнением поиска входящие данные проходят валидацию через функцию `getValidatedData`, что обеспечивает корректность и безопасность дальнейшей работы. Далее из параметров формируется динамический SQL-запрос, через:

- ✓ определение логики объединения условий, строгая через AND, либо свободная через OR;
- ✓ добавление фильтров по популярности, типу товаров и атрибутам;
- ✓ учитывание смещения для пагинации выдачи.

Все фильтры объединяются в одну строку запроса, дополняясь соответствующими параметрами для защиты от SQL-инъекций. Фрагмент кода, реализующий данный функционал представлен ниже:

```
$strictType = empty($data["strict_search"]) ? "OR" :  
"AND";  
$whereSQL = "WHERE ";  
$lastKey = array_key_last($data);  
foreach ($data as $key => $value) {  
    $whereSQL .= $value["sql"];  
    $whereParams[] = $value["param"];  
    if ($key != $lastKey) {  
        $whereSQL .= " $strictType ";  
    }  
}
```

Получив сформированные условия поиска, механизм запускает функцию `getItems`, которая отвечает за выборку товаров из базы данных. Эта функция принимает на вход параметры фильтрации, сортировки, лимит и смещение. В зависимости от заданных условий поиска она формирует SQL-

запрос к базе данных, выбирает соответствующий набор товаров и возвращает их в виде массива.

Если пользователь авторизован на сайте, дополнительно осуществляется получение списка товаров в корзине и в избранном для текущего пользователя. Это позволяет отображать актуальную информацию о состоянии корзины и избранных товаров прямо в каталоге. Реализация этой логики подробно описана в следующем блоке кода:

```
$items = makeSelectQuery("SELECT * FROM `items`  
$whereSQL ORDER BY $orderBy LIMIT $limit OFFSET $offset",  
$whereParams, false);  
if (isUserAuth()) {  
    $userItems = makeSelectQuery("SELECT * FROM  
`baskets` WHERE `users_id_orders` = ?", [getUserID()],  
false);  
    $favoritesItems = makeSelectQuery("SELECT * FROM  
`favorites` WHERE `users_id_favorites` = ?",  
[getUserID()], false);  
}  
$result = getItemsHTML($items, $userItems,  
$favoritesItems);
```

Далее вызывается функция `getItemsHTML`, которая играет ключевую роль в формировании визуальной части страницы каталога. Эта функция отвечает за создание структурированного HTML-кода для каждого найденного товара, чтобы пользователь получил наглядное и удобное представление ассортимента.

В процессе работы `getItemsHTML` перебирает массив товаров, полученный из базы данных, и для каждого элемента формирует отдельный HTML-блок. В каждом таком блоке обязательно присутствуют основные сведения о товаре. Это обеспечивает наглядное представление каждого продукта, делая каталог более удобным для пользователя.

Особое внимание уделяется интерактивным элементам. Для каждого товара формируются кнопки или соответствующие ссылки, позволяющие добавить товар в корзину и избранное. При этом для авторизованных пользователей может отображаться дополнительная информация, например,

если товар уже находится в их корзине или избранном, кнопка меняет вид, что снижает вероятность ошибочного нажатия и делает интерфейс более дружелюбным и информативным. Реализация данной функции представлена ниже:

```
foreach ($items as $item) {
    $result .= "<span class='item' data-
id='$item[id_items]' data-count='$item[count_items]'">
        <a href
=' /user/aboutItem.php?id_item=$item[id_items]'
class='item-link'>
            <p>№ $item[id_items]</p>
            <img src='$item["image_items"]' class='item-
image'>
            <p class='item-name'>$item[name_items],
$item[cost_items]</p></a>
            getBuyBasketHTML ($userItems, $item,
$item[favoritesItems]);</span>";
}
```

В зависимости от результата поиска, пользователю возвращается либо массив найденных товаров, либо сообщение о том, что ничего не найдено, если результаты отсутствуют. Кроме того, он легко интегрируется в общую архитектуру интернет-магазина и масштабируется при необходимости добавления новых параметров поиска. В результате такой гибкой и масштабируемой системы поиска пользователь получает максимально персонализированный и удобный опыт взаимодействия с каталогом, что способствует повышению его удовлетворенности и уверенному выбору товаров.

После того как пользователь находит интересующий товар, следующим логичным шагом становится переход к детальной информации о конкретном продукте. Страница товара предназначена для отображения подробных данных о выбранном продукте: его характеристиках, описании, изображениях и других важных деталях. Кроме того, она предоставляет пользователю доступ к ключевым функциям, таким как просмотр характеристик, добавление отзывов, добавление товара в корзину или избранное и ознакомление с

похожими товарами, что способствует более глубокому взаимодействию и принятию решения о покупке.

При открытии страницы система проверяет корректность передачи идентификатора товара через GET-параметр. Если параметр отсутствует или содержит неправильное значение, пользователь автоматически перенаправляется на главную страницу сайта. Эта проверка обеспечивает безопасность и целостность работы страницы, предотвращая ошибки и возможные неправильные обращения. Данная реализация описана в следующем блоке кода:

```
if (empty($_GET["id_item"])) {  
    redirect();  
}
```

После проверки наличия корректного идентификатора, информация о товаре загружается из базы данных через составной SQL-запрос. Этот запрос предназначен для получения всей необходимой информации о выбранном товаре, такой как его характеристики, описание, изображения, цена и наличие на складе. Использование такого подхода позволяет эффективно извлечь все нужные данные одним запросом, обеспечивая быстрое и точное отображение информации на странице.

Для администраторов страницы предусмотрен специальный функционал, который позволяет быстро и удобно управлять информацией о текущем товаре. В частности, для них отображается ссылка, ведущая к панели редактирования деталей продукта. Это значительно облегчает процесс внесения изменений, позволяет быстро обновлять описание, характеристики или изображения товара без необходимости искать отдельные страницы или обращаться к базе данных вручную.

Такой подход ускоряет работу по управлению каталогом и повышает эффективность работы администратора. В реализации для этого проверяется роль пользователя с помощью функции `isAdmin`. Если пользователь является администратором, на страницу добавляется HTML-код с ссылкой, которая открывает страницу редактирования товара и передает в нее идентификатор

текущего продукта через GET-параметр. Ниже представлена реализация данной логики:

```
if (isAdmin()) $editItemHTML .= "<a  
href='/admin/editItem.php?id_item=$_GET[id_item]'  
class='button'></a>";
```

На этой странице также реализован продуманный модуль работы с отзывами и дополнительной информацией о товаре, который значительно повышает удобство использования и расширяет функциональные возможности страницы. Для каждого продукта автоматически рассчитывается средний рейтинг, основанный на всех ранее оставленных оценках. Эта оценка отображается в виде числа, что помогает новым покупателям быстро понять общее качество товара и способствует формированию доверия к продукту.

Для авторизованных пользователей на странице доступна специальная форма, позволяющая оставить текстовый отзыв и выставить индивидуальную оценку. Такой подход стимулирует активное участие посетителей интернет-магазина, что помогает наполнять базу отзывов реальными мнениями и создает более доверительную атмосферу вокруг товаров. Этот блок кода реализует добавление нового комментария в базу данных:

```
$validatedData = getValidatedData($comment);  
$dateTime = date("y-m-d H:i:s");  
$isSuccess = makeSafeQuery("INSERT INTO  
`comments`(`users_id_comments`,`text_comments`,`rating_  
comments`,`date_add_comments`,`items_id_comments`)  
VALUES(?,?,?,?,?)", [getUserID(), $comment["text_comments"],  
$comment["rating_comments"], $dateTime, $comment["id_i  
tems"]]);  
$comment = makeSelectQuery("SELECT * FROM `comments`  
JOIN `users` ON `users_id_comments` = `id_users` WHERE  
`date_add_comments` = ? AND `id_users` = ?", [$dateTime,  
getUserID()], false);
```

Далее вызывается функция `getCommentsHTML`, которая отвечает за формирование HTML-разметки для отображения комментариев на странице. Эта функция берет массив комментариев, полученный с сервера, и последовательно обрабатывает каждое сообщение в цикле. В процессе обработки для каждого комментария выполняются несколько важных

действий, направленных на создание полноценного и авторитетного отображения в интерфейсе:

- ✓ если комментарий принадлежит текущему пользователю, то рядом с ним автоматически появляется кнопка для удаления этого отзыва, что обеспечивает удобство контроля контента;
- ✓ для отображения рейтинга используется SVG-иконка звезды, которая повторяется в цикле в зависимости от выставленной оценки, создавая визуальное представление оценки в виде звёздочных символов;
- ✓ для каждого комментария выводится аватар пользователя, его имя, дата публикации комментария и текст отзыва, что делает вывод информативным и приятным для восприятия.

После обработки всех комментариев сформированная HTML-разметка возвращается из функции. В самом конце эта разметка выводится на страницу, создавая конечный визуальный массив отзывов. Такой подход позволяет динамично и красиво отображать отзывы, обеспечивая хорошую интерактивность и удобство использования страницы для пользователей. Для выполнения данной задачи используется следующий фрагмент кода, который реализует необходимую логику:

```
foreach ($comments as $comment) {  
    if ($comment["users_id_comments"] ==  
getUserID()) $deleteButton .= "<button class='button'  
data-id='{$comment[id_comments]}></button>";  
    $commentsHTML .= "<div class='comment'>  
    <img src='{$comment[avatar_users]}>  
    <p class='username'>{$comment[name_users]}</p>  
    <p class='date'>{$comment[date_add_comments]}</p>  
    <p class='text'>{$comment[text_comments]}</p>  
    <p class='rating'>{$comment[rating_comments]}</p>  
    $deleteButton</div>";  
}
```

Комментарии сортируются по дате создания, благодаря этому свежие и наиболее актуальные мнения всегда отображаются первыми, что помогает посетителям сразу ознакомиться с последней информацией о продукте. Такой подход позволяет потенциальным покупателям быстро понять, что думают

другие клиенты, узнать о достоинствах товара и возможных недостатках прямо от тех, кто уже совершил покупку. Социальное доказательство в виде отзывов значительно мотивирует посетителей принять решение о покупке, поскольку видят реальный опыт других людей и уменьшают свои сомнения. Это способствует формированию доверия к товару и повышает вероятность совершения покупки.

Формирование блока с похожими товарами реализуется по аналогии с выводом товаров на главной странице сайта. Для этого применяется универсальная функция `getItems`, которая принимает параметры для определения позиции начала выборки, условий фильтрации и дополнительных настроек. Реализация этой логики описана в следующем блоке кода:

```
getItems(0, "WHERE (items_type_id_items = ? OR  
properties_id_attributes = ?) AND id_items != ?",  
[_GET["id_item"]], false, 20)
```

В этом вызове передается условие, позволяющее выбрать товары той же категории или с совпадающими характеристиками. Одновременно исключается текущий товар из выборки, чтобы отображались только похожие товары. В результате выполнения функции формируется массив похожих товаров, который затем передается в функцию формирования HTML. Именно так на странице продукта наполняется секция с похожими товарами.

Эта логика обеспечивает быстрый и эффективный механизм рекомендации товаров, делает оформление единообразным и адаптируемым для разных разделов сайта. Использование одной универсальной функции для выборки и отображения упрощает поддержку и изменение кода: при необходимости изменить условия фильтрации, сортировку или лимит результатов, достаточно скорректировать параметры вызова функции, не затрагивая основную структуру шаблона.

Такой подход способствует тому, что пользователь дольше остается на сайте, исследует больше товаров и лучше ориентируется в ассортименте. В итоге, страница товара превращается в гибкий, информационно насыщенный и цельный элемент интернет-магазина, который способен обеспечить

удобство покупателя, повысить конверсию и упростить административное управление содержимым.

2.4 Обзор и управление wybranными товарами и заказами

Пользовательский интерфейс позволяет добавлять товары в избранное, что делает взаимодействие с интернет-магазином более удобным и персонализированным. Эта функция особенно полезна для тех, кто хочет сохранить интересующие позиции и быстро возвращаться к ним позднее без необходимости повторного поиска. Таким образом, у пользователя формируется персональный список любимых товаров, который сохраняется на сервере и привязан к его аккаунту.

Для обеспечения приватности и индивидуальности, доступ к этому разделу осуществляется только после авторизации пользователя. При заходе на страницу избранных товаров происходит проверка состояния входа, если пользователь не авторизован, его автоматически перенаправляют на страницу входа. Это гарантирует, что личные предпочтения каждого пользователя остаются закрытыми и недоступными другим лицам.

В реализации функционала используется универсальная функция `getItems`, которая отображает товары в различных разделах сайта, включая данную страницу, главную страницу и страницу о товаре. Эта функция принимает параметры фильтрации, что позволяет гибко формировать список элементов в зависимости от текущего режима отображения.

В данном случае используется SQL-запрос с JOIN-операцией, объединяющий таблицы товаров и избранных, что обеспечивает выборку только тех позиций, которые добавлены конкретным пользователем. Этот подход позволяет не дублировать код и использовать единую функцию для разных сценариев отображения.

Результаты работы функции преобразуются в HTML-структуру карточек товаров. Каждая карточка содержит изображение, название, описание и кнопку

для удаления из избранного, что улучшает пользовательский опыт и дает возможность управлять списком прямо на странице.

Для отображения списка используется условие, если в избранном нет товаров, пользователь видит сообщение о пустом списке, в противном случае список товаров. Такой подход делает интерфейс интуитивно понятным и визуально аккуратным. Реализация описана в следующем блоке кода:

```
$itemsHTML = getItem(0, "JOIN `favorites` ON  
`items_id_favorites` = `id_items` WHERE  
`users_id_favorites` = ?", [getUserID()]);  
<section class="items">  
    <?= $itemsHTML == "" ? "<h1  
class='notfound'></h1>" : $itemsHTML ?>  
</section>
```

Таким образом, страница о избранных товарах превращается в удобный и эффективный инструмент для быстрого доступа к товарам, представляющим особый интерес для пользователя. Наличие персонального списка способствует увеличению вовлеченности и способствует формированию привычки регулярно возвращаться на сайт для просмотра и покупки любимых позиций. Это создает ощущение удобства и заботы о пользовательских предпочтениях, повышая общее впечатление от использования интернет-магазина.

Благодаря тщательной продуманной архитектуре системы, операции добавления и удаления товаров в этот раздел выполняются быстро и без задержек, что положительно сказывается на пользовательском опыте. Динамическое отображение данных подстраивается под индивидуальные предпочтения каждого клиента, делая взаимодействие с сайтом максимально персонализированным. Такой подход не только способствует удержанию клиентов, но и стимулирует повторные покупки, так как нужные товары всегда находятся под рукой, что повышает вероятность их приобретения.

Внутри этого контекста особенно важна страница корзины, которая реализует полноценный цикл покупки, начиная с добавления товаров и заканчивая финальным оформлением заказа. Это один из ключевых элементов

интерфейса, который обеспечивает пользователю возможность контролировать свои покупки, отслеживать текущие позиции и просматривать историю предыдущих заказов. Такой подход повышает удобство использования сайта и стимулирует к завершению покупок, делая процесс максимально прозрачным и комфортным.

После авторизации пользователь получает доступ к своей личной корзине, а также к истории прошлых заказов, что существенно облегчает повторные покупки и управление заказами.

При этом, для обеспечения безопасности и защиты личных данных фильтрация информации собирает данные только по текущему аккаунту пользователя. Это значит, что никто, кроме владельца аккаунта, не может просматривать чужие заказы или историю покупок, что важно для защиты приватности. Ниже показана функция `getBasketHTML`, которая генерирует HTML-код для отображения содержимого корзины и истории заказов:

```
foreach ($basket as $item) {
    if (empty($item["datetime_buy_orders"])) {
        $currentHTML .= getItemHTML($item);
        $currentCost += $item["cost_items"] *
$item["count_baskets"];
    } else {
        if ($idOrder != $item["id_orders"]) $historyCost
= 0;
        $historyHTML .= getItemHTML($item);
        $historyCost += $item["cost_items"] *
$item["count_baskets"];
    }
}
```

Функция `getItemHTML`, показанная выше, предназначена для генерации HTML-кода для отображения отдельного товара в корзине или заказе. Она собирает информацию о товаре и создает для него структурированный блок, который можно вставлять в страницу.

Эта функция помогает делать интерфейс более визуально привлекательным и информативным, автоматически формируя карточки товаров на основе данных из базы данных. Благодаря таким подходам,

пользователю проще ориентироваться в своем заказе и принимать решения о покупке. Реализация описана в следующем блоке кода:

```
return "<a href='/user/aboutItem.php?id
=$item[id]' class='item'>
    <img src='$item[image_items]' class='image'>
    <p class='name'>$item[name_items]</p>
    <p class='count'>$item[count_baskets]</p>
    <p class='cost'>$item[cost_items]p</p>
</a>";
```

После того как пользователь просмотрел содержимое своей корзины, он может перейти к финальному шагу, нажав кнопку «Оформить заказ». После этого открывается форма, в которой нужно указать адрес доставки, а также можно оставить комментарий для курьера.

Инновационной особенностью системы оформления доставки является автоматическая проверка возможности доставки в выбранное пользователем время. В процессе формирования списка доступных интервалов система анализирует наличие свободных курьеров. Если все курьеры заняты в этот промежуток, такой интервал времени автоматически исключается из вариантов для выбора. Реализация описана в следующем блоке кода:

```
$delivers = makeSelectQuery("SELECT COUNT(*) as
`count` FROM `users` WHERE `roles_id_users` = ?", [3],
true);
$currentTime = new DateTime()->modify("+1 hour");
$busyTimes = makeSelectQuery("SELECT * FROM `orders`
WHERE `datetime_start_orders` >= ? HAVING COUNT(*) = ?",
[$currentTime->format("Y-m-d H:i:s"),
$delivers["count"]], false);
$endTime = (clone $currentTime)->modify("+1 day")-
>setTime(0, 0);
while ($currentTime < $endTime) {
    if (!in_array($currentTime->format("Y-m-d H"),
$busyTimes)) $timesOrders[] = $currentTime->format("H");
    $currentTime->modify("+1 hour");
}
```

После того как пользователь ввёл все нужные данные и нажал кнопку оформления доставки, на сервере реализуется процесс оформления заказа пользователя. В начале данные проходят проверку валидности с помощью

функции `getValidatedData`, которая сверяет их на корректность. После этого устанавливается текущая дата и время. Следующим шагом происходит обновление записи в таблице заказов, сохраняются дата покупки, адрес, комментарий, а также запланированное время начала и конца доставки.

Таким образом, функция полностью закрывает цикл оформления заказа, что позволяет обеспечить надежность операции, правильное отображение информации и удобство для пользователя. Ниже приведён пример рабочего кода, реализующий указанную функциональность:

```
$validatedData = getValidatedData($json,
"basket.php");
$datetime = date("y-m-d H:i:s");
$isSuccess = makeSafeQuery("UPDATE `orders` SET
`status_id_orders` = ?, `datetime_buy_orders` = ?,
`address_orders` = ?, `note_orders` = ?,
`datetime_start_orders` = ?, `datetime_end_orders` = ?
WHERE `users_id_orders` = ? AND `status_id_orders` = ?",
[2, $datetime, $json["address_orders"],
$json["note_orders"],
$json["datetime_plan_orders"]["start"],
$json["datetime_plan_orders"]["end"], getUserID(), 1,]);
```

Когда пользователь завершает оформление заказа в корзине, для него становится доступна специальная кнопка, которая переводит его на страницу «жизненного пути» его заказа — от момента выбора товаров до их фактического получения. Эта страница создана для того, чтобы дать пользователю полное и прозрачное представление о ходе выполнения его заказа, повысить уровень доверия и сделать процесс более понятным и удобным.

На этой странице отображается подробный список всех товаров, входящих в заказ: с фотографиями, названиями, ценами и количеством — так пользователь может точно вспомнить, что он заказывал, убедиться в правильности выбора и контролировать состав заказа. В центре внимания находится цепочка статусов, которая показывает текущий этап выполнения заказа. Она отображается наглядно в виде списка, где текущий этап выделен

ярким цветом, чтобы пользователь сразу видел, где находится его заказ. Эта цепочка включает шесть ключевых этапов:

- ✓ в корзине — товары только добавлены в корзину;
- ✓ ожидание — заказ оформлен, но его пока не начали собирать;
- ✓ сборка — заказ начали собирать;
- ✓ доставка — все товары уже подготовлены и переданы курьеру;
- ✓ доставлено — курьер доставил товар по адресу, но пользователь ещё не подтвердил получение;
- ✓ получено — пользователь нажал на кнопку подтверждения, магазин зафиксировал успешное завершение заказа.

Система получает текущий статус заказа, которая берется из первого товара в корзине. Затем осуществляется цикл по всему списку всех возможных статусов, и для каждого статуса определяется, был ли он уже выполнен: если статус у товара меньше или равен текущему уровню, к нему применяется класс `completed`, что позволяет подсветить его как завершённый. В противном случае статус отображается без этого класса, обозначая, что он ещё не достигнут.

Когда заказ достигает финального статуса, к цепочке добавляется кнопка, которая позволяет пользователю подтвердить получение. Фрагмент кода, отвечающий за данный функционал представлен ниже:

```
$currentStatusItemInBasket =  
$itemsInBasket[0]["id_status"];  
foreach ($allStatus as $index => $status) {  
    $activeClass = $status["id_status"] <=  
$currentStatusItemInBasket ? "completed" : "";  
    $allStatusHTML .= "<p  
class='$activeClass'>$index+1$status[name_status]</p>";  
}  
if ($currentStatusItemInBasket == 5) $allStatusHTML  
.= "<button class='button receipt' data-id-  
order='$itemsInBasket[0][id_orders] '></button>";
```

Таким образом, этот кусок кода делает процесс отслеживания статусов наглядным и интерактивным, помогая пользователю сразу видеть, на каком этапе находится его заказ, и при необходимости — подтвердить получение

товара прямо на странице, делая всю процедуру прозрачной и удобной. Больше не нужно звонить в магазин и уточнять, всё прозрачно и удобно. Если заказ завис на каком-то статусе, пользователь может увидеть это и обратиться в поддержку, если потребуется, что делает такой подход процесс покупки более надежным и понятным, а самостоятельный контроль за доставкой превращает использование интернет-магазина в ещё более приятное и предсказуемое занятие.

2.5 Панель управления заказами для курьеров

После завершения оформления заказа на доставку товаров информация о заказе автоматически поступает на специализированную страницу, которая предназначена для сотрудников службы доставки. Эта страница играет ключевую роль в организации процесса выполнения заказов, предоставляя курьерам необходимую для работы информацию. Функциональные возможности данной страницы включают:

- ✓ просмотр всех новых заявок на доставку, сотрудник видит список всех недавно поступивших заказов;
- ✓ обзор содержимого заказов для каждой заявки, включающее список приобретенных товаров и их количество;
- ✓ прием заказа в работу: курьер может выбрать конкретный заказ, нажав кнопку, что означает его взятие на выполнение.

Данный функционал делает раздел мощным инструментом для работы курьеров, позволяя им получать текущий список задач в реальном времени и самостоятельно выбирать, какие заказы выполнить в первую очередь. Это способствует повышению эффективности работы службы доставки и сокращению времени обработки заявок.

На этапе обработки заказов важно реализовать механизм разграничения доступа к информации, связанной с доставкой. Только авторизованные пользователи, которым назначена роль доставщика, должны иметь возможность просматривать новую информацию о заказах и выполнять

дальнейшие действия, такие как принятие заказа в работу. Такая мера позволяет защитить персональные данные пользователей, а также предотвращает доступ к внутреннему функционалу интернет-магазина посторонних лиц.

Для реализации такого контроля на сервере используется дополнительная проверка. Сначала система определяет, авторизован ли пользователь в данный момент. Затем осуществляется проверка, входит ли учетная запись пользователя в группу доставщиков. В случае, если одно из этих условий не выполняется, пользователь автоматически перенаправляется на страницу авторизации. Это гарантирует, что доступ к служебной информации получают только сотрудники доставки, имеющие соответствующие права. Описанный механизм реализован следующим фрагментом кода:

```
if (!isUserAuth() || !isDeliver()) {  
    redirect("user/auth.php");  
}
```

После выполнения проверки роли и авторизации, на сервере выполняется SQL-запрос, который извлекает из базы данных все заказы, находящиеся в статусе ожидания обработки. Для этого используется сложный запрос, объединяющий несколько связанных таблиц: основную таблицу заказов, таблицу деталей заказа, таблицу товаров и таблицу статусов.

В результате получается полная информация о каждом заказе, включая состав приобретённых товаров, их количество, стоимость и другую важную информацию. Заказы сортируются по времени оформления, чтобы показывать в первую очередь самые свежие заявки. Самое важное — заказы группируются по времени оформления, что позволяет для каждого заказа сформировать отдельный блок. В этом блоке указывается время покупки, а также приводится детальная информация по каждому товару.

После списка товаров для каждого заказа размещается кнопка, нажав которую доставщик может взять данный заказ в работу. В случае, если новые заказы отсутствуют, системе выводится сообщение о том, что на данный

момент заказов для обработки нет. Если же есть доступные заказы, на странице список всех доступных заявок, что обеспечивает прозрачность процесса и облегчает распределение задач между доставщиками. Такой механизм способствует ускорению обработки заказов, повышению их эффективности и автоматизации логистики внутри интернет-магазина. Ниже представлена реализация данного сценария:

```
foreach ($baskets as $index => $basket) {
    if($datetimeBuy!=$basket["datetime_buy_orders"]) {
        $basketsHTML .= "<article
class='basket'><h2>$basket[datetime_buy_orders]</h2><div
class='items'>";
        $datetimeBuy = $basket["datetime_buy_orders"];
    }
    $basketsHTML .= getItemHTML($basket);
    if ($index == $lastIndexBasket || $datetimeBuy !=
null && $baskets[$index + 1]["datetime_buy_orders"] !=
$datetimeBuy) $basketsHTML .= "</div><button
class='button'data-id-order='$basket[id_orders]'">
</button></article>";
}
```

После того как доставщик принял заказ, он попадает на специально созданную страницу, где отображаются все заказы, закрепленные за ним. В начале работы страницы осуществляется выборка всех заказов, которые были приняты данным курьером из базы данных, и они сортируются по дате начала доставки, чтобы наиболее свежие заказы отображались сверху.

После этого заказы делятся на заказы, находящиеся в статусе от ожидания до принятия пользователем и выполненные заказы. В первом списке для каждого заказа отображаются важные параметры, такие как время оформления, крайний срок доставки, текущий статус, а также список товаров, входящих в заказ.

Для обеспечения эффективного и удобного управления процессом доставки для курьеров реализована система специальных кнопок, позволяющих изменять статус заказа на каждом этапе выполнения. Эти кнопки внедряются с помощью JavaScript, что обеспечивает динамичное и

интерактивное взаимодействие без необходимости перезагружать страницу. На интерфейсе отображаются кнопки, соответствующие конкретным действиям. Помимо кнопок, каждой записи присваиваются информационная подсказка и название текущего статуса, что помогает курьеру быстро ориентироваться и понимать текущий этап выполнения заказа.

При нажатии на любую кнопку с помощью JavaScript происходит отправка асинхронного запроса на сервер, который в свою очередь обновляет статус заказа в базе данных. В процессе выполнения операции кнопка отображает прогресс и после успешного завершения подтверждает изменение статуса, а также обновляет текстовые метки, чтобы пользователь видел актуальный статус. Описанный механизм реализован следующим фрагментом кода:

```
button.addEventListener("click", async () => {
  const resultData = await sendToServer({
    "server_type": "status_orders",
    "id_orders": button.dataset.idOrder,
  });
  if (resultData["status"] == "OK") {
    button.dataset.idStatus++;
    nameStatus.textContent =
`${resultData["data"]["name_status"]}${allStatus[button
.dataset.idStatus]["text"]}`;
    if (button.dataset.idStatus > 4) button.remove();
    else button.textContent =
allStatus[button.dataset.idStatus]["action"];
  }
});
```

Данный механизм позволяет курьерам управлять доставками максимально удобно и быстро, избегая ручного обновления страниц и ошибок, связанных с неправильными действиями. Такой подход повышает скорость обработки заказов и наполняет процесс контроля за доставкой гибкостью и автоматизацией.

А второй раздел отображает уже завершённые заказы. Здесь фиксируются время покупки и дата окончания заказа, а также перечень

доставленных товаров и доставщик может только просмотреть историю своих заявок.

Для корректного отображения всех актуальных заказов, закрепленных за конкретным доставщиком, используется цикл, который перебирает каждый заказ из полученного из базы данных массива. Этот процесс позволяет сгруппировать товары по отдельным заказам, а также добавить к каждому заказу всю необходимую информацию.

На каждом этапе цикл анализирует, относится ли очередной элемент к новому заказу. Если это так, создаётся новая карточка заказа, включающая заголовки с деталями и секцию для выводимых товаров. Далее автоматически добавляется каждая позиция заказа с помощью отдельной функции `getItemHtml`. Реализация подробно описана в следующем блоке кода:

```
foreach ($baskets as $index => $basket) {
    if ($basket["id_status"] <= 5) {
        $basketsCurrentHTML .= getItemHTML($basket);
        if ($datetimeBuy !=
$basket["datetime_buy_orders"]) $datetimeBuy =
$basket["datetime_buy_orders"];
    } else {
        $basketsHistoryHTML .= getItemHTML($basket);
        if ($index == $lastIndexBasket || $datetimeBuy !=
null && $baskets[$index + 1]["datetime_buy_orders"] !=
$datetimeBuy) $datetimeBuy =
$basket["datetime_buy_orders"];
    }
}
```

Как только весь состав заказа сформирован, происходит «закрытие» карточки заказа, добавляется кнопка для совершения действия, и вывод завершается. Благодаря такой структуре обеспечивается компактное и в то же время информативное отображение всех необходимых данных для работы курьера делая интерфейс максимально удобным.

Если ни одного актуального заказа ещё нет, пользователю выводится надпись о том, что у него нет ни одного заказа в процессе. Аналогично информируется, если раньше не было завершённых заказов. Таким образом,

представленный комплекс логических и интерфейсных решений обеспечивает курьерам интуитивно понятный и эффективный инструмент для контроля выполнения заказов, поэтапного отслеживания статусов и просмотра истории всех доставленных товаров. Реализация такой страницы играет ключевую роль в автоматизации процессов службы доставки интернет-магазина, повышая прозрачность и упрощая взаимодействие персонала с системой.

2.6 Административная панель для управления

Административная панель предназначена для управления ключевыми сущностями интернет-магазина, обеспечивая административному персоналу возможность оперативного и удобного редактирования данных, а также контроля за функционированием платформы.

На данной странице администратору предоставляется интуитивно понятный интерфейс с набором инструментов, позволяющих быстро обновлять информацию о важных разделах. Благодаря логичной структуре и удобной организации элементов пользователю легко ориентироваться и получать доступ к нужным разделам, что значительно повышает эффективность работы и сокращает время на выполнение повседневных задач.

Доступ к административной панели строго ограничен и осуществляется только авторизованным пользователям с правами администратора. Для обеспечения безопасности реализована серверная логика проверки прав входа, при которой любой пользователь, не обладающий необходимыми правами, автоматически перенаправляется на главную страницу. Такой механизм исключает возможность несанкционированного доступа к управленческому интерфейсу и защищает критические данные платформы. Реализованная система проверки прав представлена в следующем фрагменте кода:

```
if (!isAdmin()) {  
    redirect();  
}
```

Основная страница панели управления состоит из набора навигационных кнопок, каждая из которых ведёт к отдельному разделу с

возможностью просмотра, редактирования или добавления данных. Здесь администратор может выполнить следующие действия:

- ✓ перейти к созданию товаров;
- ✓ открыть список свойств товаров и изменять их;
- ✓ редактировать типы товаров;
- ✓ управлять списком пользователей и их данными.

Такой подход обеспечивает централизованное управление всеми базовыми сущностями и позволяет быстро реагировать на изменения: обновлять ассортимент, гибко менять или корректировать свойства позиций в соответствии с бизнес-процессами.

Раздел создания товаров в административной панели предназначен для быстрого и удобного внесения новых товаров в базу данных интернет-магазина. Этот интерфейс задуман таким образом, чтобы максимально упростить работу администратора и обеспечить корректность данных, а также полноту информации о каждом товаре.

Основным механизмом работы является обработка формы через POST-запрос. После отправки формы сервер сначала выполняет валидацию всех введенных данных, проверяет их на корректность и полноту. В случае успеха, подготовленные данные вставляются в таблицы базы данных. Для этого используется специальная функция `getValidatedData`, которая собирает и проверяет все поля формы и возвращает результат, а также подготовленный SQL-запрос и параметры для безопасного выполнения. Если валидация прошла успешно, сервер вставляет базовые параметры товара в таблицу товаров, используя сформированный SQL-запрос. После этого извлекается ID добавленного товара, чтобы связать его с дополнительными характеристиками.

Администратор может добавить произвольное количество свойств товара, например, цвет или размер, каждый из которых связан с множеством возможных значений. Для этого генерируются динамические SQL-запросы на вставку с помощью подготовленных выражений. Если возникла ошибка, то

система сохраняет введенные данные и информирует администратора о неудаче. Для выполнения данной задачи используется следующий фрагмент, который реализует необходимую логику:

```
$validatedData=getValidatedData(array_merge($_POST,
$_FILES));
$tempSQL=getInsertSQL($validatedData["data"]);
$isSucceedItem=makeSafeQuery("INSERT INTO `items`
($tempSQL[sql]) VALUES ($tempSQL[question])", $tempSQL["pa
rams"]);
$id = $link->lastInsertId();
$itemProperties=$validatedData["data"]["items_prope
rties"];
foreach ($itemProperties as $property) {
    $sql.="INSERT INTO `items_properties`
(`items_id_items_properties`,`attributes_id_items_prope
rties`) VALUES (?, ?);";
    array_push($params,$id,$property["id_attributes"]);
}
makeSafeQuery($sql, $params);
```

Сам интерфейс формы реализован в виде набора полей, которые динамически создаются с помощью скриптов, основываясь на данных, полученных из базы данных. При этом в системе автоматически генерируются HTML-элементы для выбора различных свойств и атрибутов, а также для их связки и настройки.

Такой подход позволяет обеспечить гибкость и масштабируемость формы, так как все поля и их параметры формируются в реальном времени, отражая актуальные данные, которые могут меняться без необходимости ручной правки кода интерфейса.

Для удобства администратора система реализует механизм отображения подсказок и уведомлений о результатах выполненных действий. Так, при успешном добавлении товара администратор видит подтверждающее сообщение, а в случае ошибок — уведомления о том, что именно пошло не так, что помогает быстро ориентироваться в процессе работы.

Все операции по добавлению, редактированию или удалению данных сопровождаются безопасной обработкой информации. Для этого используются

подготовленные запросы, которые гарантируют защиту от таких угроз, как SQL-инъекции, и позволяют сохранять целостность базы данных. Таким образом, все вводимые данные проходят проверку и обрабатываются с учетом правильной экранизации и безопасности. Ниже представлена реализация динамического создания полей из базы данных:

```
$allProperties = makeSelectQuery("SELECT * FROM
`properties`", [], false);
foreach ($allProperties as $option)
    $allPropertiesHTML .= "<option
value='$option[id_properties]'$>$option[name_properties]
</option>";
$allAttributes = makeSelectQuery("SELECT * FROM
`attributes`", [], false);
foreach ($allAttributes as $attribute)
    $allAttributesHTML .= "<label
class='hidden'$>$attribute[value_attributes]<input
class='input' type='checkbox'
value='$attribute[id_attributes]' data-
name='attributes_select_value' data-is-insert-
server='1'$></label>";
```

Раздел изменения товара в административной панели интернет-магазина предназначен для редактирования всех основных данных о существующем товаре, таких как название, цена, количество, тип, описание, изображение, а также связанных свойств и атрибутов. Доступ к этому разделу строго ограничен и проверяется в начале выполнения кода.

Для загрузки текущих данных о выбранном товаре используется его уникальный идентификатор, после чего формируется форма, предварительно заполненная этими сведениями, а также списками всех доступных свойств и атрибутов. Когда администратор заполняет и отправляет форму, вводимые значения проходят первичную валидацию с помощью функции `getValidatedData`. В случае успешной проверки происходит подготовка к обновлению информации в базе данных.

После проверки корректности данных формируется набор SQL-запросов для синхронизации свойств и атрибутов: удаляются те свойства и атрибуты, которые отмечены как удалённые, и добавляются новые, если они ещё

отсутствуют в базе. Также формируется SQL-запрос на обновление основной информации о товаре, включающей название, цену, количество и остальные параметры. Все подготовленные SQL-инструкции выполняются в рамках транзакции, чтобы обеспечить целостность данных. После успешного завершения операции пользователю выводится сообщение о том, что изменение выполнено успешно, либо сообщается об ошибке, если возникли проблемы во время выполнения.

В конце все действия завершаются перенаправлением на страницу товара для повторного отображения обновлённых сведений и сообщения о результате. Ниже приводится пример реализации данной логики, включающий механизм загрузки данных, валидацию, подготовку SQL-запросов и обработку результатов:

```
foreach ($itemProperties as $property) {
    if ($property["type"] == "remove") $sql .=
"DELETE FROM `items_properties` WHERE
`attributes_id_items_properties` = ? AND
`items_id_items_properties` = ?;";
    else $sql .= "INSERT INTO `items_properties`
(`items_id_items_properties`,
`attributes_id_items_properties`) VALUES (?, ?);";
}
$result = getUpdateSQL($validatedData["data"]);
if ($result["sql"] != "") $sql .= "UPDATE `items`
SET $result[sql] WHERE `id_items` = ?;";
makeSafeQuery($sql, $params);
```

В итоге, процесс изменения товара в административной панели представляет собой последовательную и надёжную процедуру. Благодаря этому подходу обеспечивается целостность данных и удобство последующего управления каталогом.

Страница редактирования пользователей предназначена для того, чтобы администратор мог управлять учетными записями пользователей интернет-магазина. Доступ к этому разделу строго ограничен и проверяется в начале выполнения кода, так же, как и в других разделах административной панели.

В верхней части страницы расположена форма поиска, позволяющая фильтровать пользователей по почте или по статусу блокировки. Этот поиск реализован таким образом, что не требует полной перезагрузки страницы, что значительно повышает удобство использования. После отправки формы результаты поиска отображаются в основной секции страницы.

Основной вывод информации осуществляется с помощью функции `getUsers`, которая выбирает из базы данных всех пользователей, кроме главного администратора системы. Для каждого пользователя формируется блок, в котором отображаются его имя, почта, текущий статус и роль. Это позволяет администратору быстро получать всю необходимую информацию и вносить изменения по мере необходимости. Ниже приведена реализация этого механизма:

```
$users = makeSelectQuery("SELECT * FROM `users`  
$where ORDER BY `id_users` DESC", $params, false);  
foreach ($users as $user) {  
    $usersHTML .= "<div class='user'>  
        <p>    $user[name_users]    |    $user[email_users]    |  
$user[is_banned_users] === 1</p>  
        <p class='user-role'>$user[name_roles]</span></p>  
        <button class=action>$action</button></div>";  
    }  
}
```

Рядом с карточками каждого пользователя расположены кнопки управления, которые позволяют быстро и удобно выполнять различные операции:

- ✓ сделать доставщиком или убрать — меняет роль пользователя;
- ✓ заблокировать или разблокировать — изменяет статус блокировки;
- ✓ сделать специалистом поддержки или убрать эту роль;
- ✓ удалить — полностью удалить профиль пользователя.

Все эти действия реализованы через AJAX-запросы, что значительно ускоряет работу и исключает необходимость перезагрузки страницы. Благодаря динамической загрузке и обновлению данных, администратор сразу видит результат своих действий.

Такой интерфейс позволяет оперативно реагировать на любые ситуации, например, блокировать нарушителя, повышать статус курьера или полностью удалять аккаунт. Этот подход обеспечивает гибкость, безопасность и простоту управления учетными записями, что особенно важно для современных онлайн-платформ

Разделы редактирования свойств и типов товаров реализуются через один универсальный файл с единым интерфейсом для добавления и редактирования записей в различных таблицах административной панели. В зависимости от значения GET-параметра, администратор может выбрать работу с таблицей свойств или типов товаров, либо добавлять новые записи. Такой подход позволяет избежать повторения похожего кода для каждой сущности и обеспечивает централизованный механизм обработки. Перед выполнением операций скрипт проверяет три условия:

- ✓ обладает ли текущий пользователь административными правами;
- ✓ существует ли GET-параметр;
- ✓ содержит ли запрос допустимое имя таблицы из списка разрешённых.

Если параметры не соответствуют требованиям, происходит перенаправление пользователя на другую страницу — это обеспечивает защиту данных. Выбор таблицы происходит динамически, если GET-параметр совпадает с одним из допустимых значений, дальнейшие действия строятся с выбранной таблицей.

При отправке формы методом POST скрипт ожидает получить данные, необходимые для либо добавления новой записи, либо редактирования существующей в выбранной таблице. Входные данные, проходя через функцию `getValidatedData` фильтруются и валидируются, чтобы гарантировать их корректность и безопасность, предотвращая возможные SQL-инъекции и другие уязвимости.

Если данные успешно прошли проверку, скрипт автоматически формирует SQL-запрос для вставки новой записи с помощью функции

getInsertSQL или для обновления существующей с помощью getUpdateSQL, что значительно упрощает работу и снижает риск ошибок, связанных с ручным написанием SQL-кода. После выполнения основной операции в текущую сессию заносится сообщение о результате — успешном завершении или возникшей ошибке. Такое сообщение хранится во временной переменной, чтобы его можно было отобразить на следующей загрузке страницы, информируя администратора о результатах его действия. Таким образом, вся процедура построена так, чтобы быть максимально безопасной и удобной, автоматически обрабатывать данные и предоставлять актуальную обратную связь. Ниже представлен код, реализующий этот процесс:

```
if ($_GET["type"] == "add") {
    $result = getInsertSQL($validatedData["data"]);
    makeSafeQuery("INSERT INTO `{$tableName}`
($result[sql]) VALUES ($result[question])",
$result["params"]);
} else if ($_GET["type"] == "update") {
    $result = getUpdateSQL($validatedData["data"]);
    makeSafeQuery("UPDATE `{$tableName}` SET
$result[sql] WHERE `id_{$tableName}` = ?",
$result["params"]);
}
```

Если в таблицу свойств вносится новое значение или обновляется существующее, система предусматривает работу с вложенными атрибутами. Это означает, что при добавлении или изменении свойства автоматически обрабатывается массив связанных вложенных атрибутов. Такой механизм позволяет поддерживать структуру данных гибкой и многогранной, а также обеспечивает целостность связанных записей.

Процесс обработки вложенных атрибутов реализуется с помощью цикла, который перебирает массив данных и для каждого элемента выполняет соответствующие операции — вставку новой записи или обновление существующей. В результате получается последовательность связанных данных, которые строго соответствуют структуре и позволяют удобно

управлять расширенными характеристиками товаров. Реализация подробно описана в следующем блоке кода:

```
foreach ($attributes as $attribute) {
    if (empty($attribute["id_attributes"])) {
        $result = getInsertSQL($attribute);
        $sql     .= "INSERT INTO `attributes`
($result[sql]) VALUES ($result[question]);";
    } else {
        $result = getUpdateSQL($attribute);
        $sql     .= "UPDATE `attributes` SET
$result[sql] WHERE `id_attributes` = ?;";
    }
}
makeSafeQuery($sql, $params);
```

Финальный этап работы скрипта — это создание HTML-разметки для административного интерфейса, которая строится на основе структуры выбранной таблицы. Именно на этом шаге динамически формируются формы для добавления новых записей и для редактирования существующих, создаются:

- ✓ формы добавления новых записей;
- ✓ списки разделов для редактирования каждой из уже существующих записей с их атрибутами;
- ✓ кнопки для удаления записей и динамические кнопки для добавления новых атрибутов или свойств;
- ✓ шаблоны для новых вложенных форм.

Таким образом данный подход обеспечивает универсальность, так как один и тот же файл служит точкой для редактирования любых таблиц справочников, благодаря чему структура форм адаптируется автоматически под любые таблицы с помощью получения метаданных.

Благодаря динамическому получению информации о связанных сущностях и полях, административное управление данными становится быстрым, безопасным и очень гибким, исключая лишнюю рутину и ручное программирование для каждого раздела. Это значительно упрощает сопровождение кода, а также позволяет расширять схемы данных и

функциональность системы по мере необходимости, не переписывая логику для каждого конкретного раздела.

2.7 Многофункциональная платформа поддержки

В рамках разработки проекта особое внимание уделялось созданию системы, которая объединяет удобство, функциональность и простоту в использовании для обеих сторон — покупателей и службы поддержки. В результате была реализована многофункциональная страница поддержки, которая становится надежным инструментом для быстрого и эффективного взаимодействия.

Эта платформа обеспечивает не только возможность оперативно получать ответы на вопросы и решать возникающие проблемы, но и предоставляет полный набор функций для управления обращениями, что способствует укреплению доверия. Данный раздел предусматривает четкое разделение ролей на обычных пользователей и сотрудников службы поддержки, что позволяет выстроить структурированную и прозрачную систему взаимодействия.

Клиенты имеют возможность самостоятельно инициировать новые обращения, в которых они могут максимально подробно описать свои проблемы, столкнувшиеся затруднения, задать интересующие вопросы либо высказать предложения по улучшению сервиса магазина. Такой подход способствует более полному и своевременному сбору обратной связи, позволяя специалистам поддержки оперативно реагировать на реальные потребности аудитории.

Пользователь и специалист по поддержке могут обмениваться сообщениями, уточнять детали, предоставлять инструкции и рекомендации. Вся история переписки системно фиксируется, что позволяет легко контролировать процесс, сохранять важные детали и избегать потерь информации. Такой подход обеспечивает прозрачность и удобство

коммуникации, повышая доверие клиентов к магазину и стимулируя их к повторным покупкам. Данная реализация, представленная ниже:

```
$validatedData=getValidatedData($json,"support.php"
);
$datetime = date("Y-m-d H:i:s");
$sql = getInsertSQL($validatedData["data"]);
$isSuccess = makeSafeQuery("INSERT INTO `supports`
($sql[sql]) VALUES ($sql[question])", $sql["params"]);
$message = ["name" => $userInfo["name_users"],
"text"      => $validatedData["data"]["text_supports"],
"date"      => dateformat($datetime), "image"      =>
$validatedData["data"]["image_supports"]];
```

Затем, в функции getMessageHTML, происходит процесс получения HTML-кода для отображения сообщения в интерфейсе поддержки. Эта функция отвечает за формирование правильной и аккуратной визуальной составляющей каждого сообщения, обеспечивая его читаемость и удобство восприятия для пользователя и сотрудника службы поддержки. Реализация описана в следующем блоке кода:

```
return "<div class='message $who'>
    <p class='message-name'>$message[name]</p>
    <p class='message-text'>$message[text]</p>
    <p class='message-date'>$message[date]</p>
    <img src='$message[image]'></div>";
```

Кроме того, пользователи имеют возможность закрывать диалоги по мере полного решения своих вопросов или в случае, когда обращение перестает быть актуальным. Эта функциональность способствует поддержанию порядка на платформе, делая систему более структурированной и удобной для всех участников. Когда обращение закрыто, оно перестает отображаться в верху списка запросов, что позволяет менеджерам поддержки сосредоточиться только на текущих, нерешённых вопросах.

Такой подход значительно облегчает работу службы поддержки, ускоряет обработку новых обращений и повышает общую эффективность системы. Пользователи, в свою очередь, чувствуют большую контроль над своими вопросами и могут легко управлять своими обращениями без лишних

усилий. Ниже представлен фрагмент кода, реализующий механизм закрытия диалога:

```
$validatedData =
getValidatedData(["talks_id_supports" => $idTalks],
"support.php");
if (! $validatedData["isCorrect"])
setAnswer("FAIL");
$isSuccess = makeSafeQuery("UPDATE `talks` SET
`is_end_talks` = ?, `datetime_end_user_talks` = ? WHERE
`id_talks` = ?", [1, date("y-m-d H:i:s"), $idTalks]);
setAnswer($isSuccess ? "OK" : "FAIL");
```

Обратная связь от специалистов поддержки организована через специально разработанный интерфейс, который отображает все входящие обращения, позволяя оперативно отслеживать их статус, тему и историю сообщений. Каждый диалог сопровождается уникальным номером — это идентификатор, благодаря которому сотрудник может быстро найти нужное обращение, а также вести его контроль и управление. Такой подход значительно упрощает навигацию и помогает избежать путаницы при работе с большим количеством обращений.

На этой специально разработанной административной странице отображаются все новые обращения, которые ещё не были приняты в работу. Это помогает моментально выявлять свежие запросы и обеспечивать оперативное реагирование. В каждом обращении есть кнопка, которая позволяет назначить специалиста. После нажатия этой кнопки выбранный сотрудник закрепляется за конкретным обращением и получает возможность вести диалог с клиентом.

Такой механизм способствует равномерному распределению нагрузки между сотрудниками, ускоряет обработку запросов и обеспечивает индивидуальный подход к каждому клиенту, что существенно повышает эффективность работы поддержки и общий уровень сервиса интернет-магазина, а также создаёт у клиента ощущение внимательного и профессионального сервиса. Ниже приведён пример рабочего кода, где

происходит закрепление специалиста за обращением по нажатию соответствующей кнопки:

```
$validatedData = getValidatedData(["id_talks" =>
$idTalks], "support.php");
if (! $validatedData["isCorrect"])
setAnswer("FAIL");
$check=makeSelectQuery("SELECT`users_support_talks`
FROM `talks` WHERE `id_talks` = ?", [$idTalks], true);
if ($check== "FAIL" || $check["users_support_talks"]
!= null) setAnswer("FAIL");
$isSuccess = makeSafeQuery("UPDATE `talks` SET
`users_support_talks` = ?,
`datetime_accept_support_talks` = ? WHERE `id_talks` =
?", [getUserID(), date("y-m-d H:i:s"), $idTalks]);
setAnswer($isSuccess ? "OK" : "FAIL");
```

Такая схема организации работы поддержки позволяет достичь нескольких важных целей:

- ✓ равномерное распределение нагрузки между операторами, каждое новое обращение автоматически назначается свободному специалисту;
- ✓ контроль текущего статуса обращений, в системе можно легко отслеживать количество открытых, в работе и уже закрытых вопросов;
- ✓ предотвращение дублирования работы, благодаря четкому закреплению операторов за каждым обращением исключается ситуация, когда одна проблема одновременно разбирается несколькими специалистами.

Все эти меры значительно улучшают качество обслуживания клиентов: сокращают время реакции на обращения, повышают точность и полноту предоставляемых решений. Это способствует формированию положительного впечатления о магазине, укреплению доверия и лояльности клиентов.

В конечном итоге, внедренная система становится надежным инструментом, который позволяет быстро и точно реагировать на любые вопросы и запросы, повышая уровень сервиса и репутацию компании. Такой подход укрепляет позиции интернет-магазина на рынке, что способствует росту положительных отзывов, созданию позитивного имиджа.

ЗАКЛЮЧЕНИЕ

Создание интернет-магазина сегодня становится необходимым шагом для любого человека, стремящегося обеспечить своему бизнесу устойчивое развитие и конкурентоспособность в условиях динамично меняющегося рынка. В ходе выполнения данной работы были рассмотрены ключевые этапы и компоненты, составляющие основу успешной онлайн-торговли. От правильного выбора технологий и платформ, до реализации безопасной и удобной системы — все эти элементы тесно связаны и требуют системного подхода.

Современные технологии позволяют реализовать масштабные решения, использующие адаптивный дизайн, автоматизированные системы аналитики и высокоуровневую защиту данных. Благодаря этому владельцы онлайн-магазинов могут одновременно охватывать широкую аудиторию, снижать операционные издержки и обеспечить клиентам высокий уровень сервиса. В то же время, внедрение новых решений требует постоянного обучения и адаптации, поскольку цифровая среда стремительно развивается и диктует новые стандарты.

Можно сказать, что разработка интернет-магазина — это комплексная задача, требующая, как технических знаний, так и стратегического мышления. В условиях жесткой конкуренции и возрастающих требований к качеству обслуживания, что возможно только при использовании современных технологий, внимание к деталям и постоянном стремлении к инновациям. Поэтому, воплощая идеи и реализуя проект, важно помнить, что именно эффективное сочетание технологий, маркетинга и пользовательского опыта помогает достигать поставленных целей и обеспечивать долгосрочный успех бизнеса в цифровом пространстве.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Алиса и Боб учатся безопасному кодированию / Учебное пособие Т. Н. Янца — Москва: ДМК Пресс, 2025, — 410с.
2. Интернет-технологии / Учебное пособие С. Р. Гуриков — Москва: Инфра-М, 2026, — 174с.
3. Компьютерные сети / Учебное пособие А. В. Кузин — Москва: Инфра-М, 2026, — 190с.
4. Основы алгоритмизации и программирования / Учебное пособие. С. Г. Канакова — Москва: Инфра-М, 2025, — 243с.
5. Основы проектирования и разработки информационных систем / Учебное пособие Л. Г. Гагарина — Москва: Инфра-М, 2026, — 211с.
6. Программное обеспечение компьютерных сетей и web-серверов / Учебное пособие Г. А. Лисьев — Москва: Инфра-М, 2026, — 145с.
7. Разработка веб-приложений на PHP 8 / Учебное пособие Д. Н. Колесниченко — Санкт-Петербург: БХВ, 2024, — 496с.
8. Системы счисления, алгоритмизация и программирование / Учебное пособие Е. П. Игнашева — Москва: Инфра-М, 2026, — 224с.
9. Современные технологии и технические средства информатизации / Учебное пособие О. В. Шишов — Москва: Инфра-М, 2026, — 462с.
10. <https://developer.mozilla.org/ru/> Интернет источник
11. <https://habr.com/ru/feed/> Интернет источник
12. <https://www.php.net/manual/ru/> Интернет источник